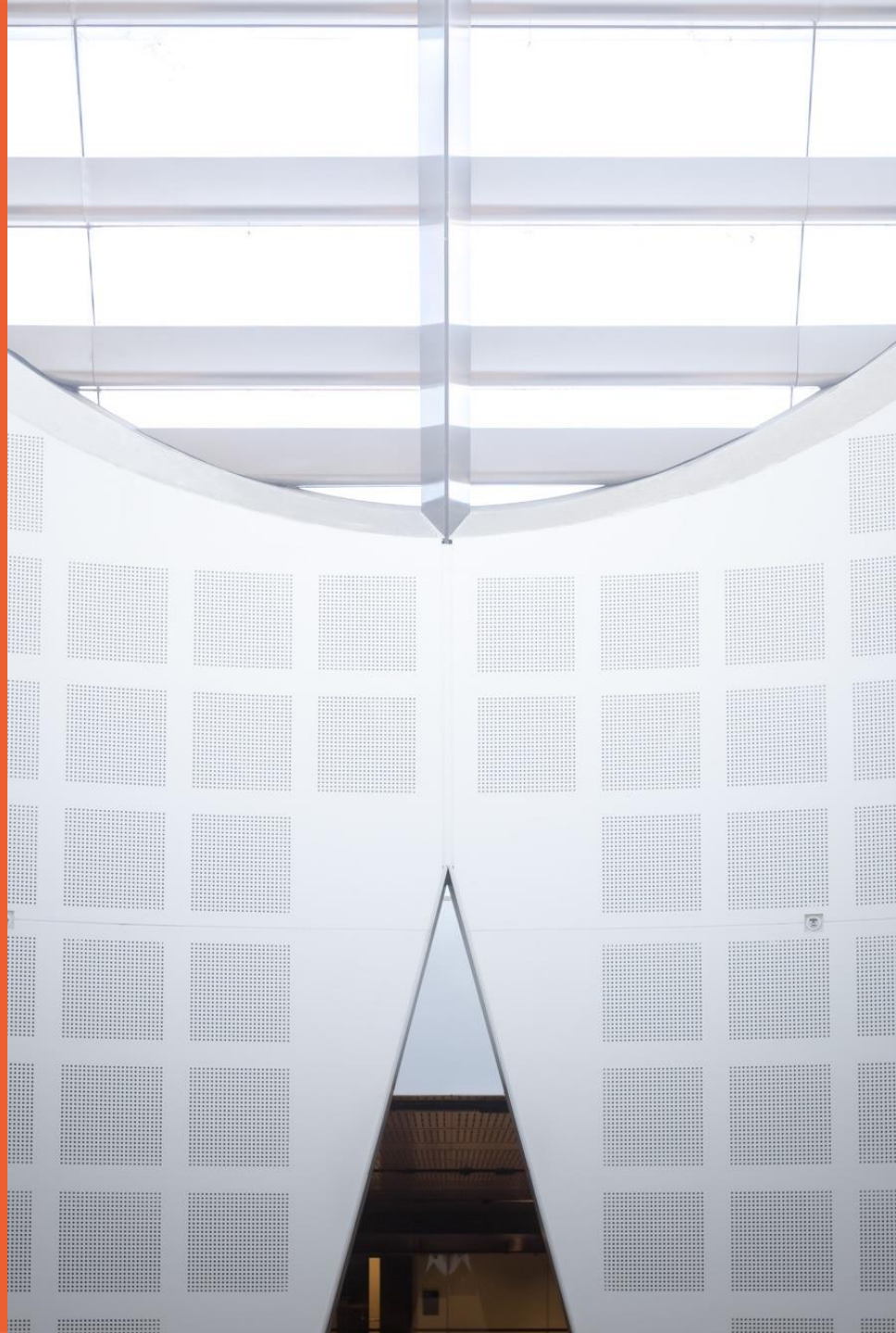
Foundations of Neural Network

Chuyang Zhou
With most of the slides prepared by A/Prof Chang Xu
School of Computer Science

THE UNIVERSITY OF SYDNEY



Outline

- What is a neural network?
- What does a neural network learn?
- How does a neural network learn?

- Optimization for Deep Model
- Parameter Initialization for Deep Model

What is a neural network?

THE UNIVERSITY OF SYDNEY

Neuron

A neuron cell

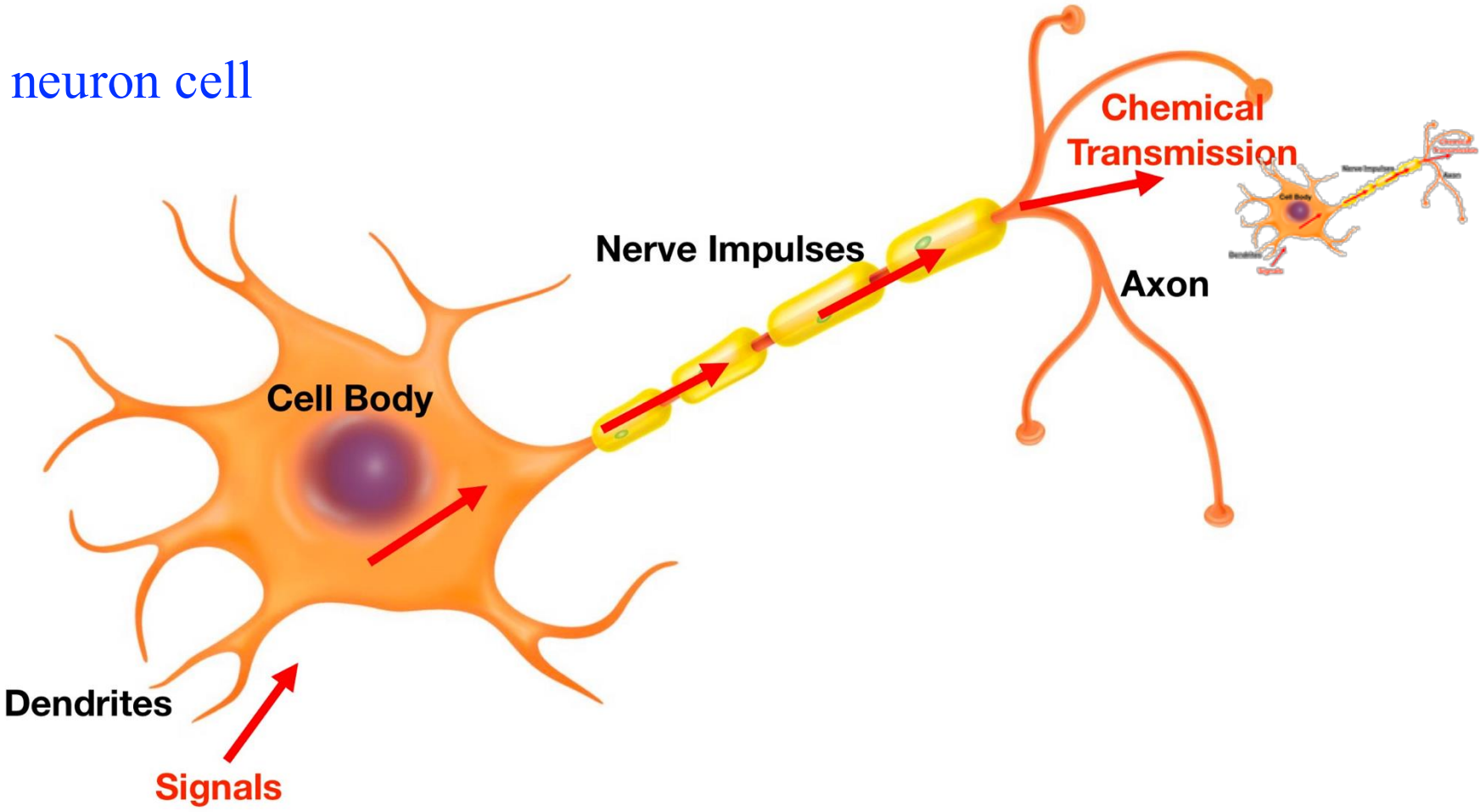
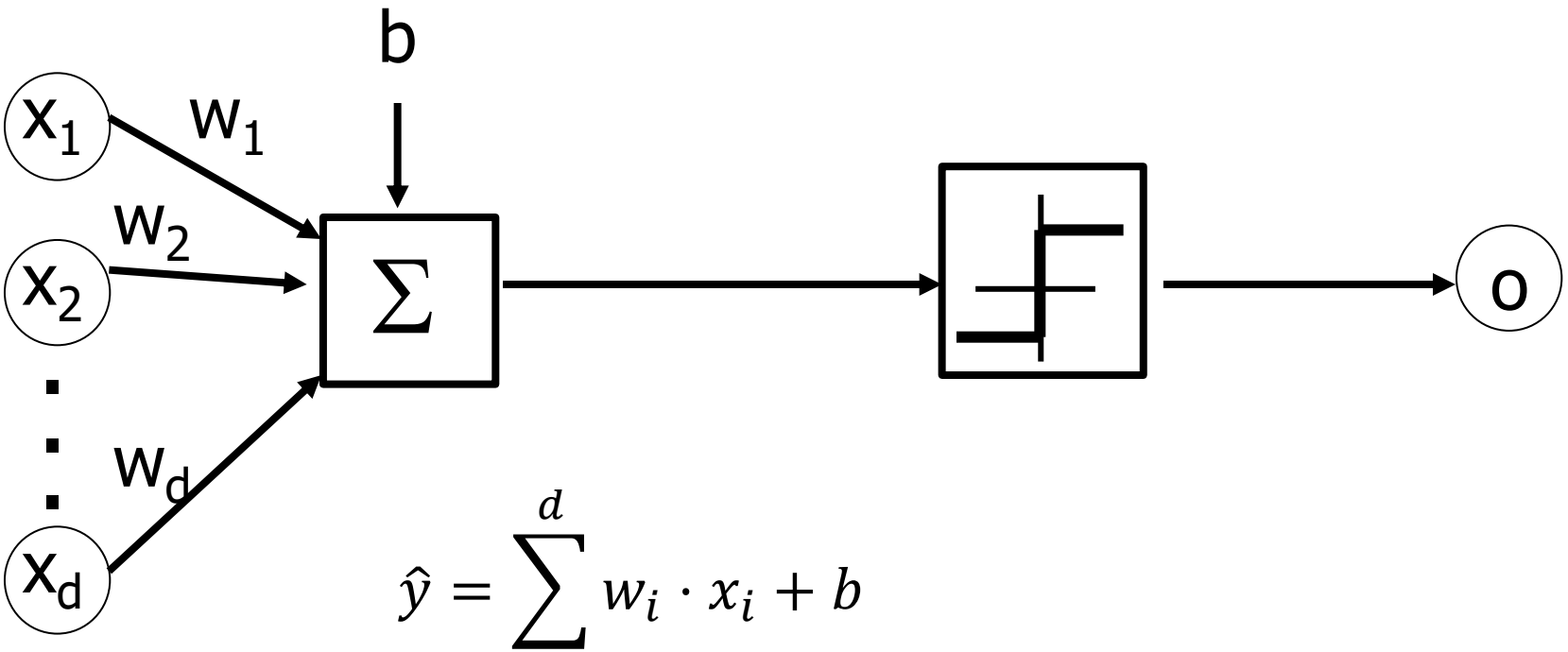


Image credit to: thinglink.com

Perceptron: the Prelude of DL

[Rosenblatt, et al. 1958]

$$o = sign(\hat{y}) = \begin{cases} +1, & \hat{y} \geq 0 \\ -1, & \hat{y} < 0 \end{cases}$$



$$\hat{y} = \sum_{i=1}^{d} w_i \cdot x_i + b$$

Perceptron

- The neuron has a real-valued output which is a weighted sum of its inputs.

weight vector

input vector

Neuron's estimate of the desired output

$$\hat{y} = \sum_{i=1}^{d} w_i x_i + b = w^T x + b$$

bias

Recall the decision rule

$$o = sign(\hat{y}) = \begin{cases} +1, & \hat{y} \geq 0 \\ -1, & \hat{y} < 0 \end{cases}$$

Now we have a linear binary classifier

$$y(w^T x + b) \geq 0$$

otherwise

$$y(w^T x + b) < 0$$

Perceptron "AI winter"

Marvin Minsky & Seymour Papert (1969). *Perceptrons*, MIT Press, Cambridge, MA.

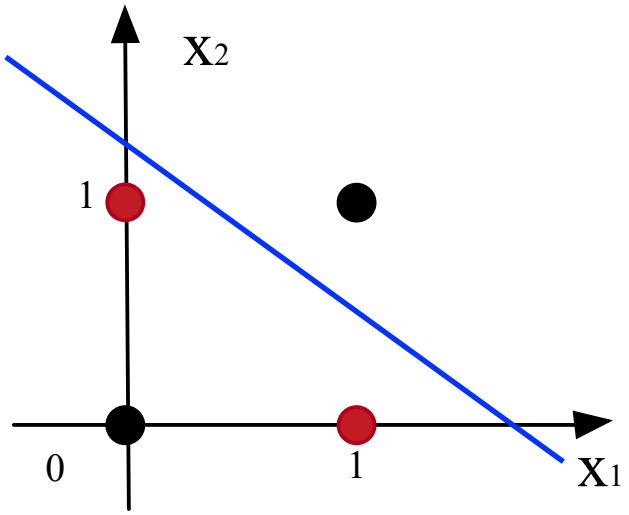
- Before long researchers had begun to discover the Perceptron's limitations.
- Unless input categories were "linearly separable", a perceptron could not learn to discriminate between them.
- Unfortunately, it appeared that many important categories were not linearly separable.
- E.g., those inputs to an XOR gate that give an output of 1 (namely 10 & 01) are not linearly separable from those that do not (00 & 11).

https://karthikvedula.com/assets/images/perceptronVis_files/plotlyPerceptronVis_circle.html

Perceptron: Limitations

– Solving XOR (exclusive-or) with Perceptron?

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



– Linear classifier cannot identify non-linear patterns.

$$w_1 x_1 + w_2 x_2 + b$$

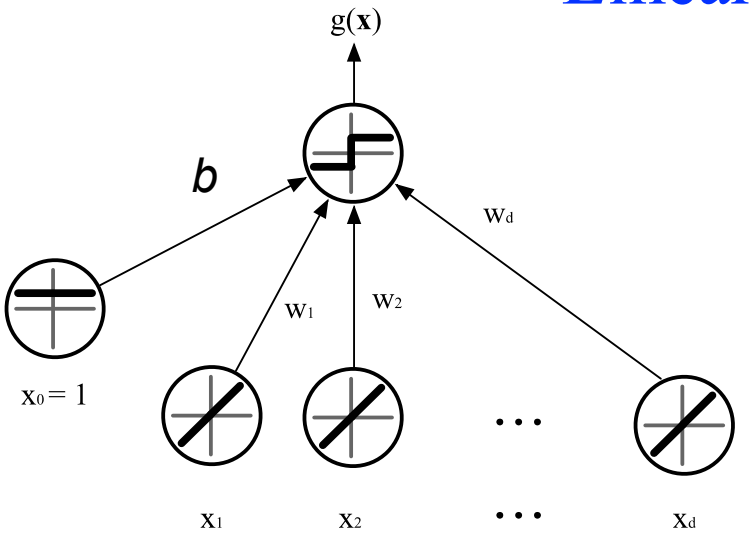
This failure caused the majority of researchers to walk away.

Breakthrough: Multi-Layer Perceptron

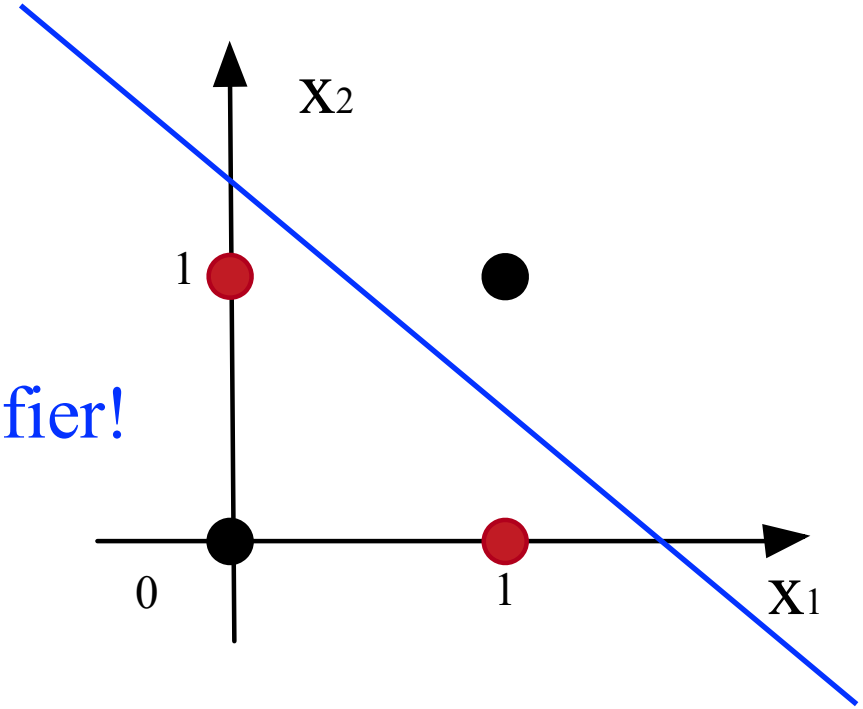
- In1986, Geoffrey Hinton, David Rumelhart, and Ronald Williams published a paper "*Learning representations by back-propagating errors*", which introduced
  - **Backpropagation**, a procedure to ***repeatedly adjust the weights*** so as to minimize the difference between actual output and desired output
  - **Hidden Layers**, which are ***neuron nodes stacked in between inputs and outputs***, allowing neural networks to learn more complicated features (such as XOR logic)

Multilayer Neural Network

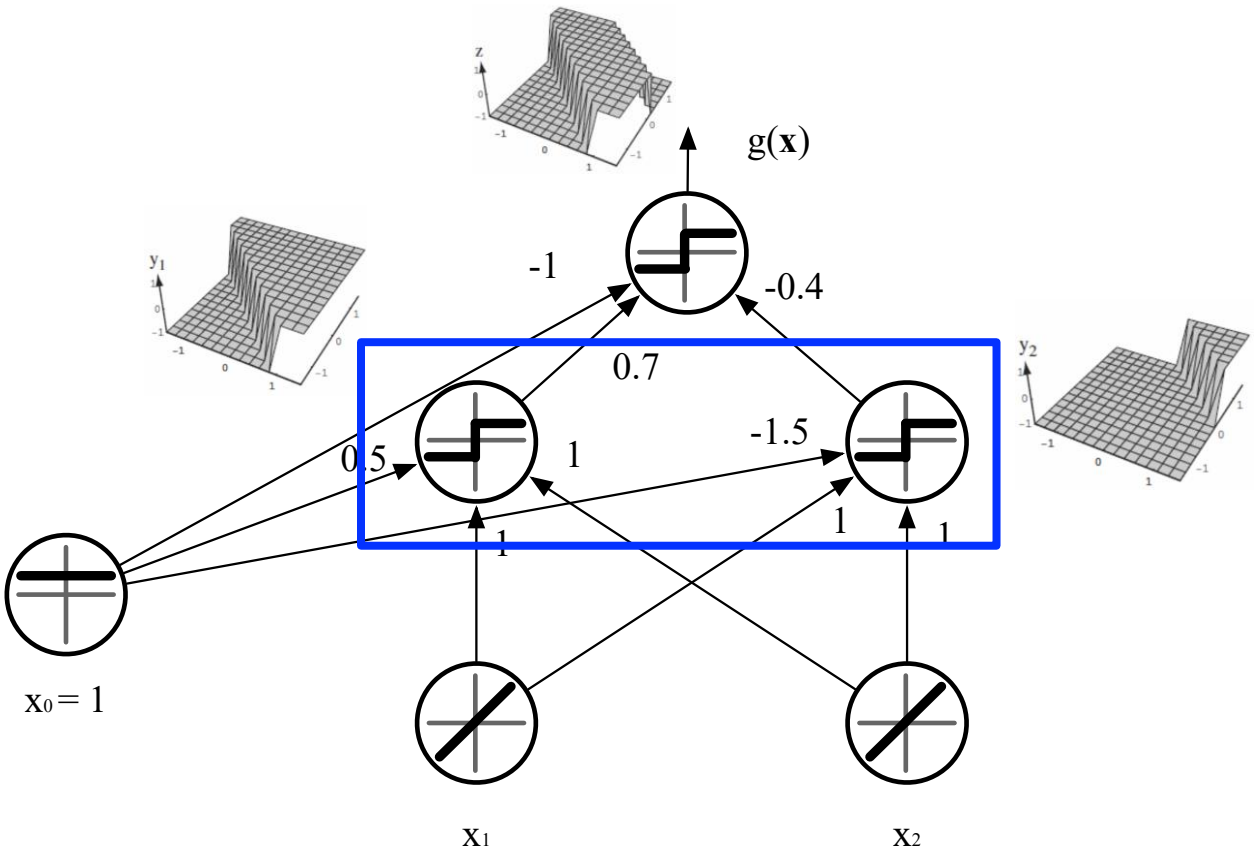
Two-layer Neural Network

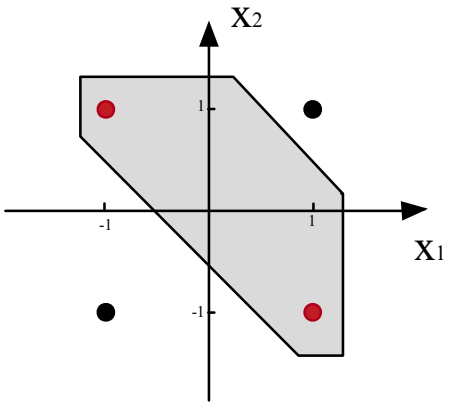


Linear classifier!



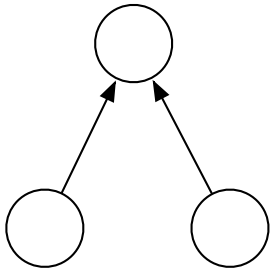
Three-layer Neural Network (with a hidden layer)

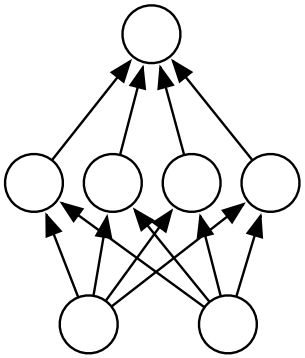


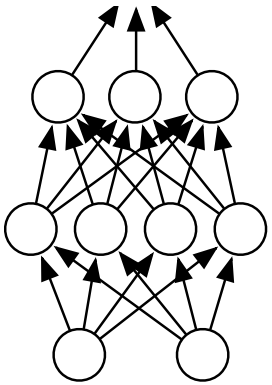


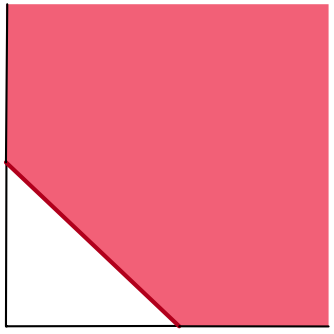
Multi-layer Neural Network

– A 2D-input example

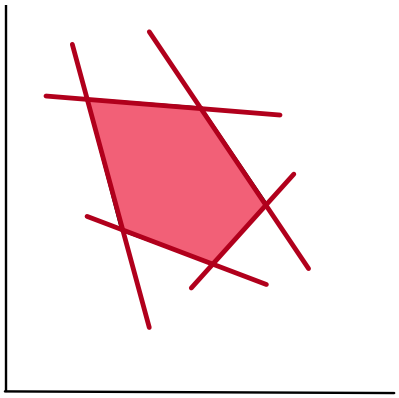




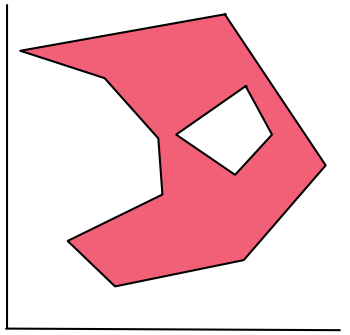




separating hyperplane



convex polygon region



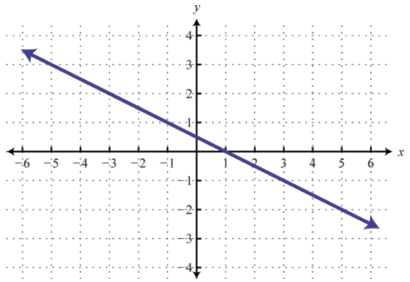
composition of polygons

Activation Functions

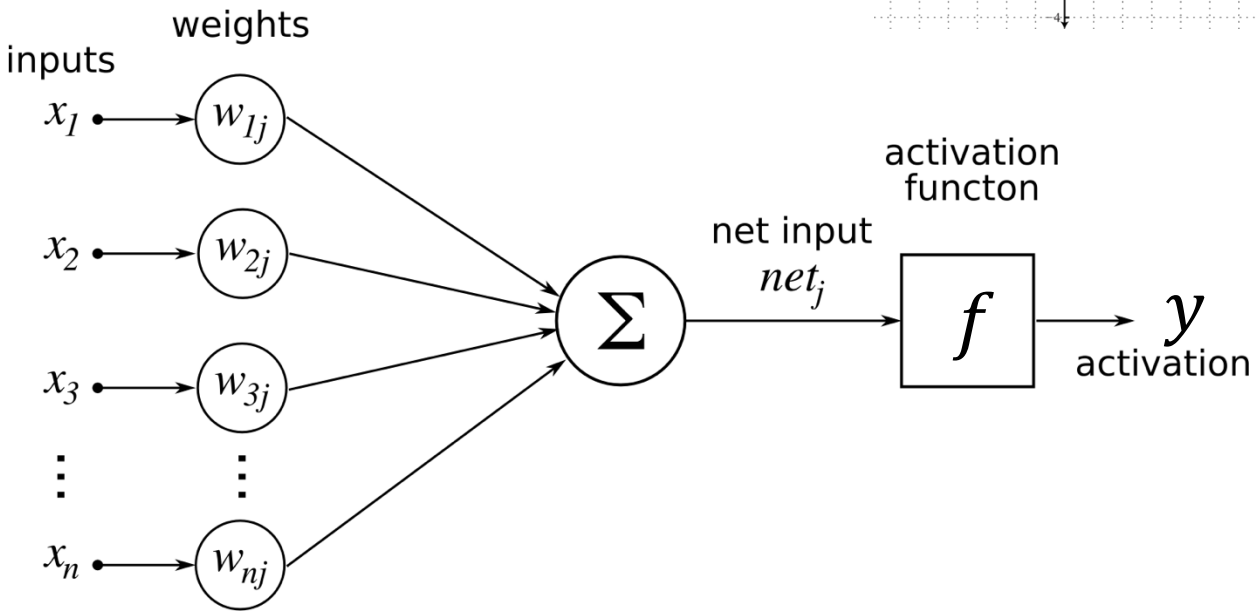
Activation functions

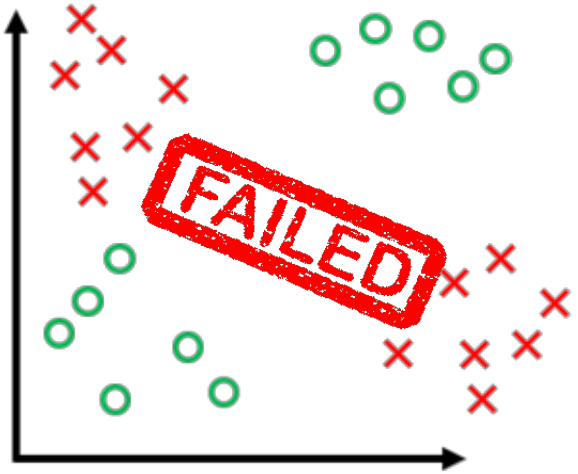
– Must be nonlinear: otherwise it is equivalent to a linear classifier

$w^T x$



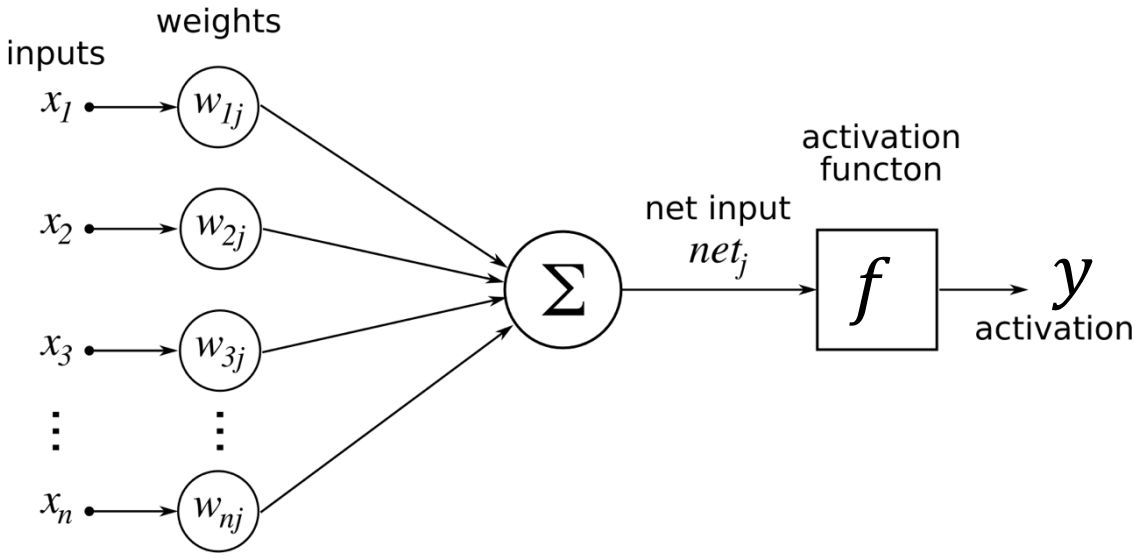
Linear function!

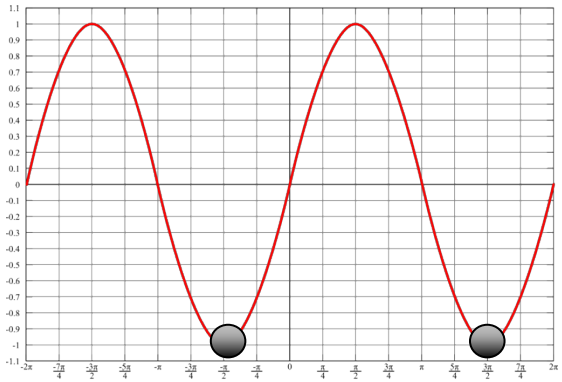




Activation functions

- Continuous and differentiable **almost everywhere**
- Monotonicity: otherwise it introduces additional local extrema in the error surface



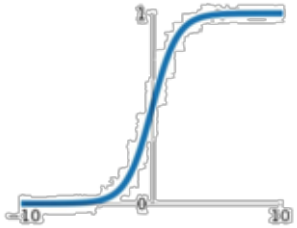


An output value might correspond to multiple input values.

Activation function $f(s)$

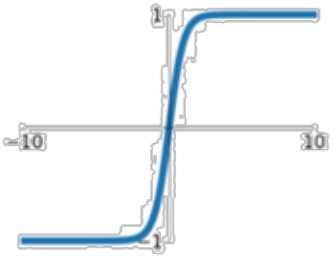
- Popular choice of activation functions (single input)
  - Sigmoid function

$$f(s) = \frac{1}{1 + e^{-s}}$$



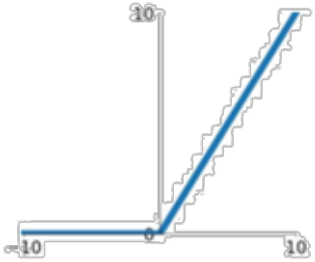
  - Tanh function: shift the center of Sigmoid to the origin

$$f(s) = \frac{e^{s} - e^{-s}}{e^{s} + e^{-s}}$$



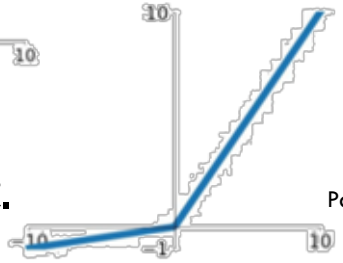
  - Rectified linear unit (ReLU)

$$f(s) = \max(0, s)$$



  - Leaky ReLU

$$f(s) = \begin{cases} s, & \text{if } s \geq 0 \\ \alpha s, & \text{if } s < 0 \end{cases}, \alpha \text{ is a small constant.}$$

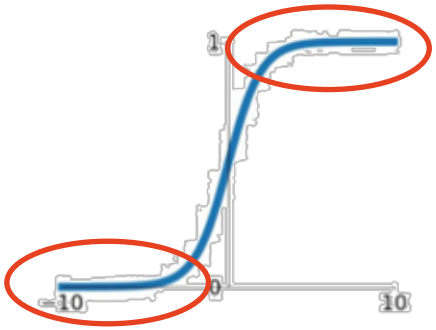


Vanishing gradient problems

– Saturation.

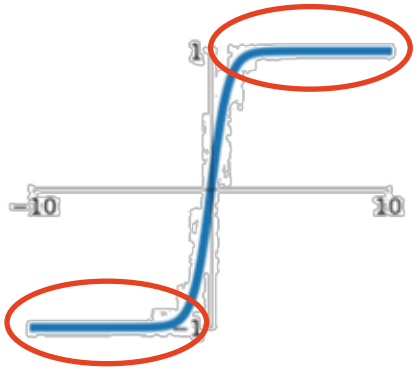
Sigmoid function

$$f(s) = \frac{1}{1 + e^{-s}}$$



Tanh function

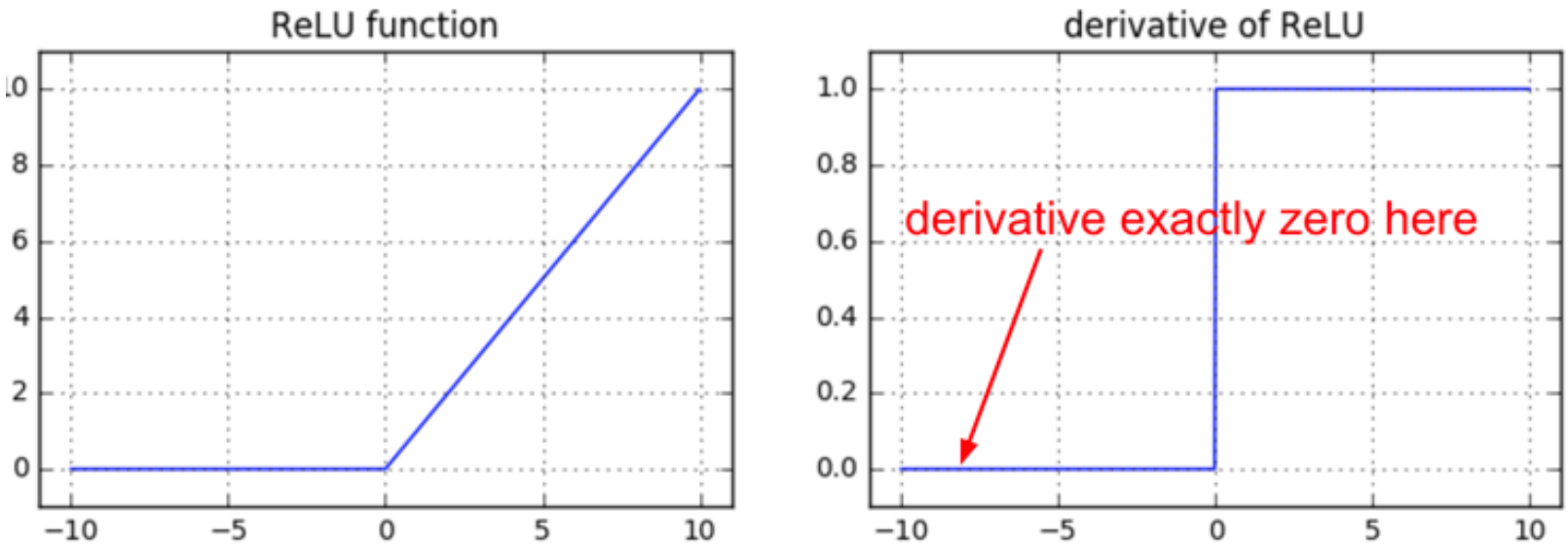
$$f(s) = \frac{e^{s} - e^{-s}}{e^{s} + e^{-s}}$$



❖ At their extremes, their derivatives are close to 0, which kills gradient and learning process

Dead ReLUs

ReLU function $f(s) = \max(0, s)$



- For negative numbers, ReLU gives 0, which means some part of neurons will not be activated and be dead.
- Avoid large learning rate and wrong weight initialization, and try Leaky ReLU, etc.

Gaussian Error Linear Unit (GELU)

$$\mathrm{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2}\left[1 + \mathrm{erf}(x/\sqrt{2})\right],$$

$\phi(x)$ is the standard Gaussian cumulative distribution function

PyTorch's exact implementation is sufficiently fast;

GELUs are used in GPT-3, BERT, and most other Transformers.

Smoother compared to ReLU, allowing it to better retain small gradient information and enhance the model's expressive power.

GELU typically achieves better convergence than ReLU and Leaky ReLU.

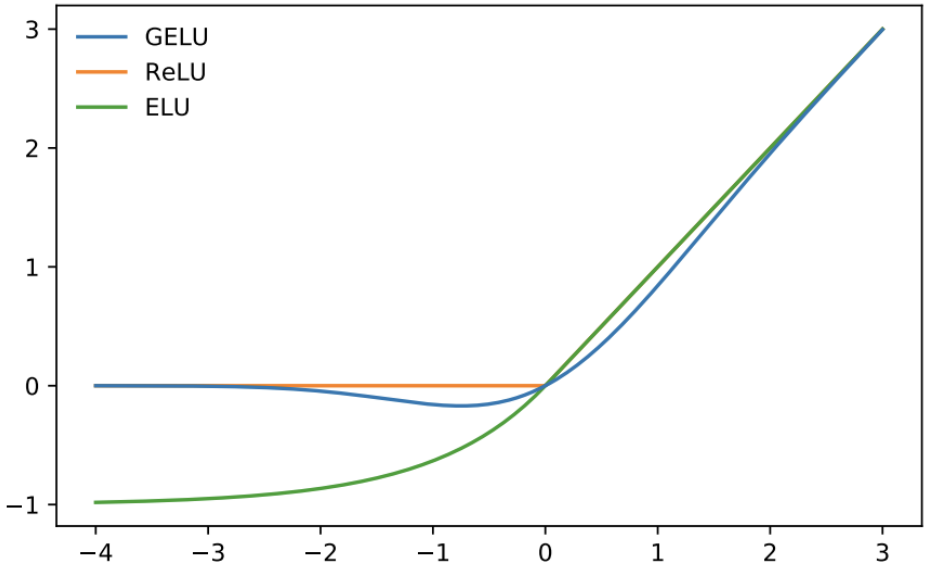
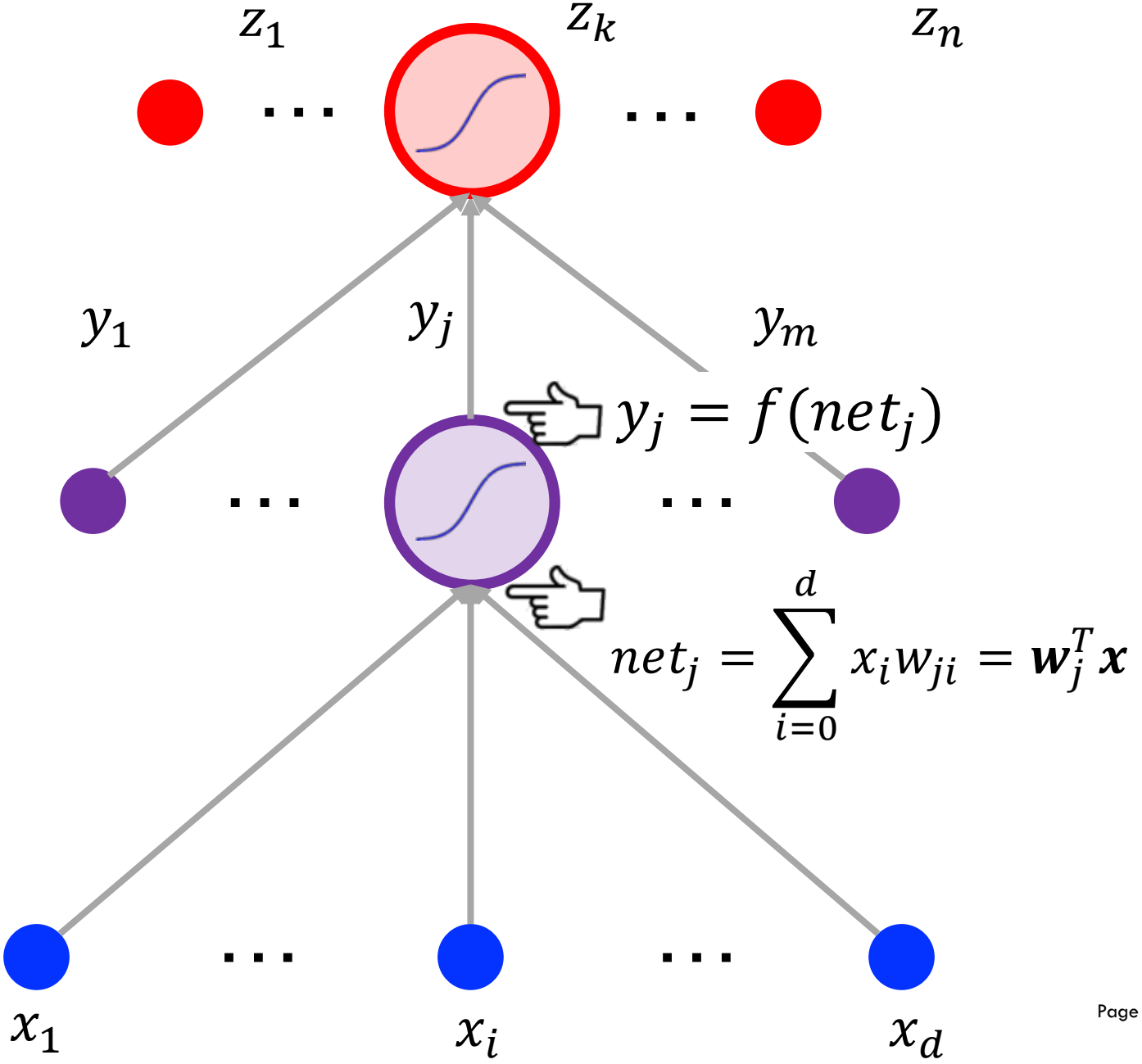
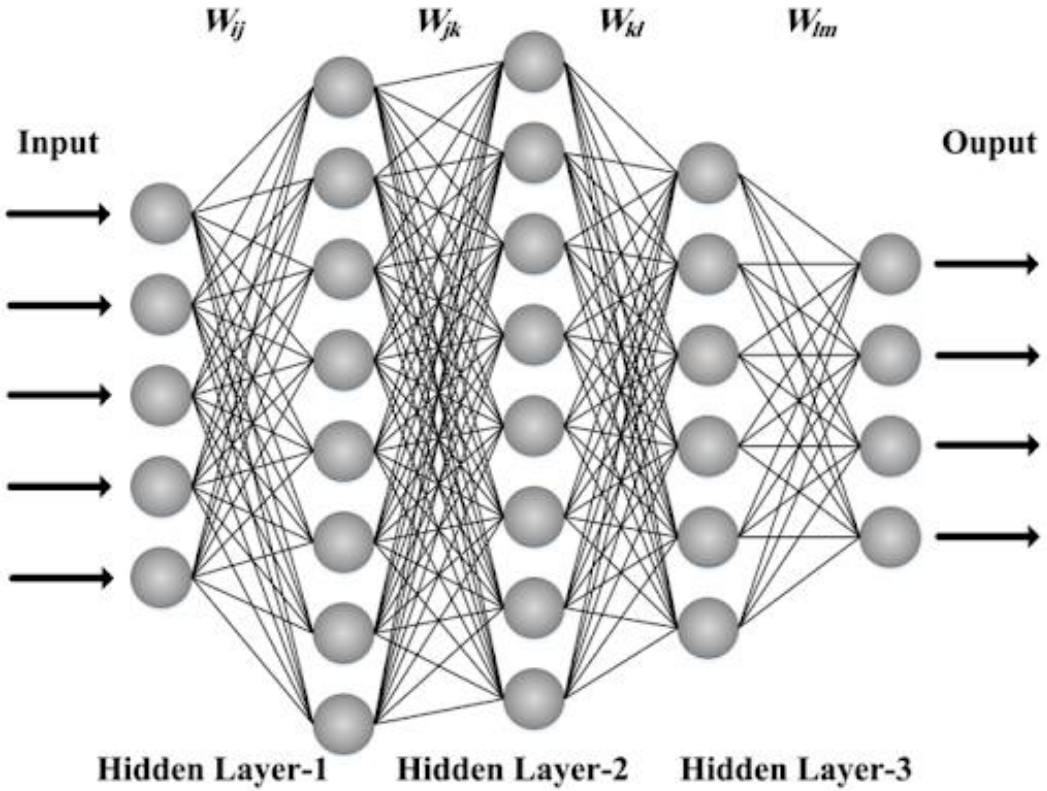


Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).

Feedforward



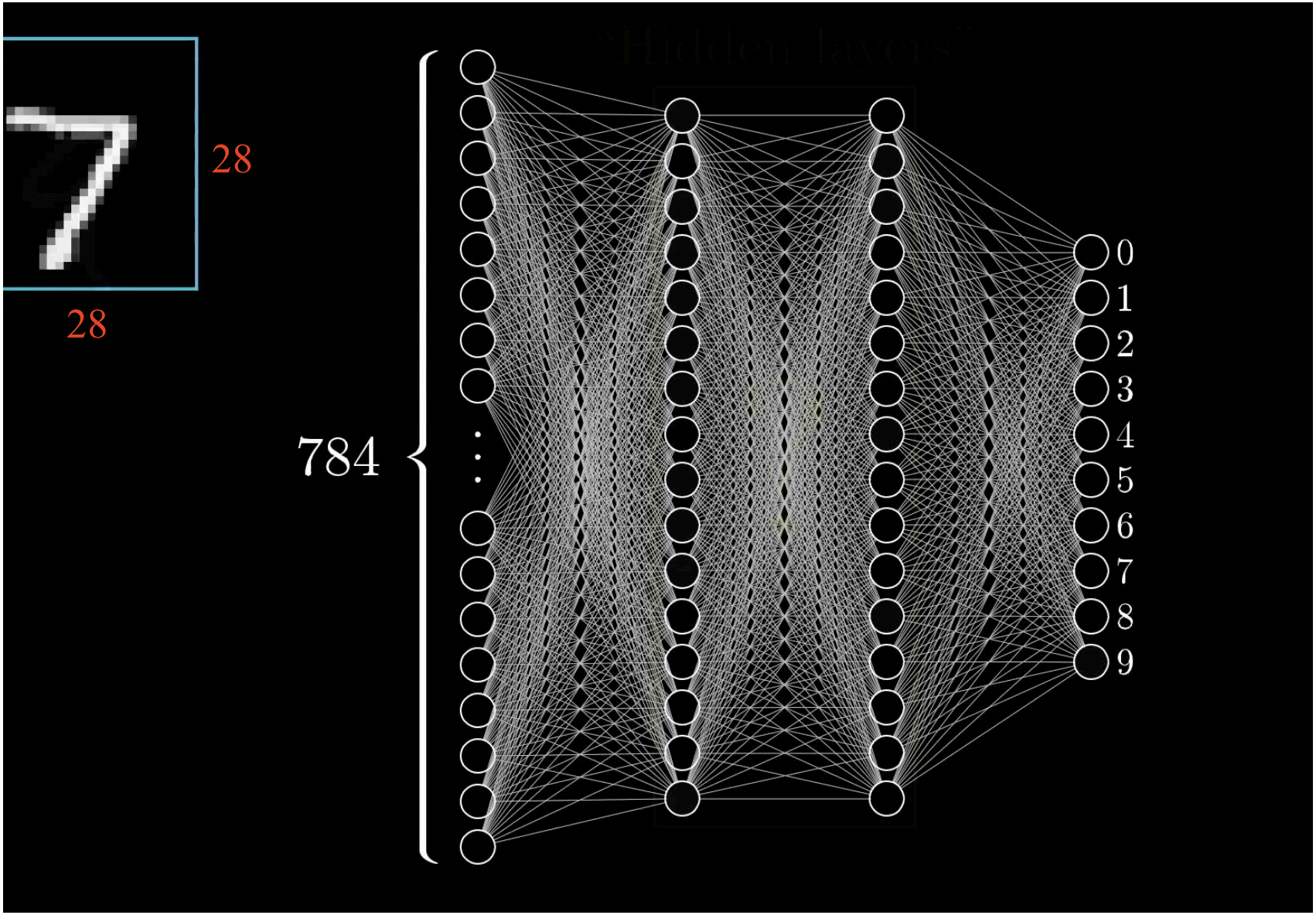
Multilayer Neural Networks



- Function composition can be described by a directed acyclic graph (hence feedforward networks)
- Depth is the maximum i in the function composition chain
- Final layer is called the output layer

What does a neural network learn?

THE UNIVERSITY OF SYDNEY


28
28
784
"Hidden layers"
0
1
2
3
4
5
6
7
8
9

The Network Is Just a Function

$$x \xrightarrow{f} y$$

Goldfish

Espresso

Dugong

Monarch Butterfly

Ladybug

A neural network learns a set of parameters (weights and biases) that define an input-to-output function minimizing a chosen loss on the training data.

Universal Approximation Theorem

Universality theorem (Hecht-Nielsen 1989): "Neural networks with a single hidden layer can be used to approximate any continuous function to any desired precision."

Let $f(\cdot)$ be a nonconstant, bounded, and monotonically-increasing continuous function. Let I_m denote the m-dimensional unit hypercube $[0,1]^m$. The space of continuous functions on I_m is denoted by $C(I_m)$. Then, given any $\varepsilon > 0$ and any function $f \in C(I_m)$, there exists an integer N, real constants $v_i, b_i \in \mathbb{R}^m$ and real vectors $w_i \in \mathbb{R}^m$, where $i = 1, \cdots, N_i$ such that we may define:

$$\hat{F}(x) = \sum_{i}^{N} v_i f(w_i^T x + b_i)$$

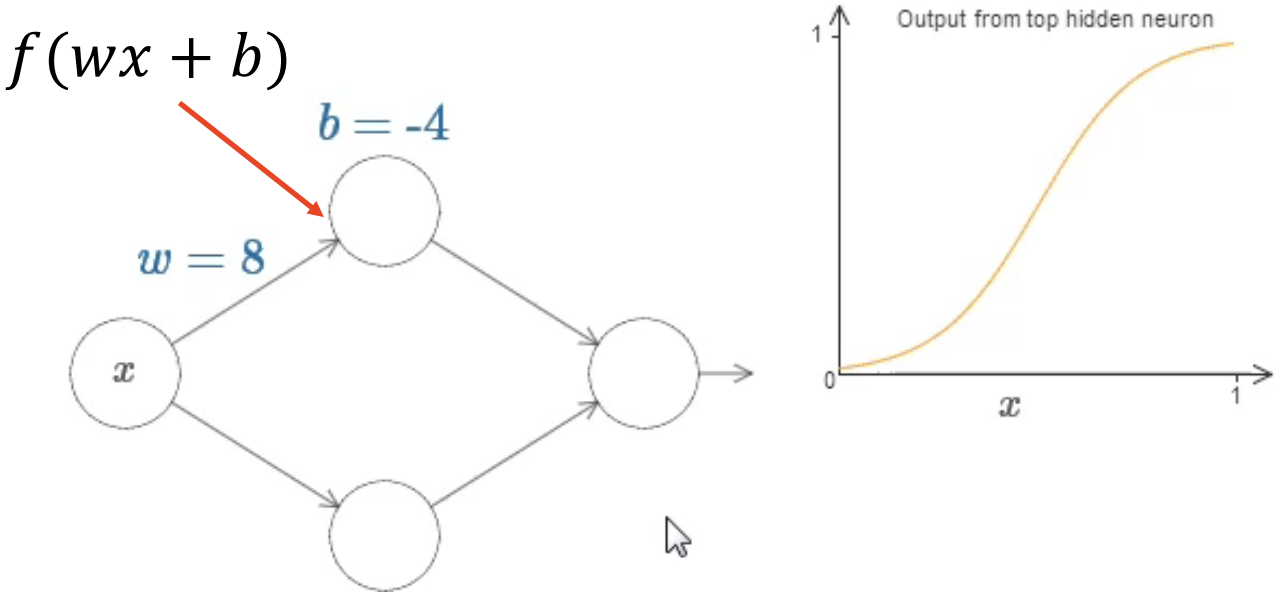
as an approximate realization f the function F where F is independent of f; that is,

$$\left|\hat{F}(x) - F(x)\right| < \varepsilon$$

for all $x \in I_m$. In other words, functions of the form $\hat{F}(x)$ are dense in $C(I_m)$.

Expressive power of a three-layer neural network

- A visual explanation on one input and one output functions



Video credit to: https://neuralnetworksanddeeplearning.com

- The weight changes the shape of the curve.
- The larger the weight, the steeper the curve.
- The bias moves the graph, but doesn't change its shape.

Expressive power of a three-layer neural network

- A visual explanation on one input and one output functions

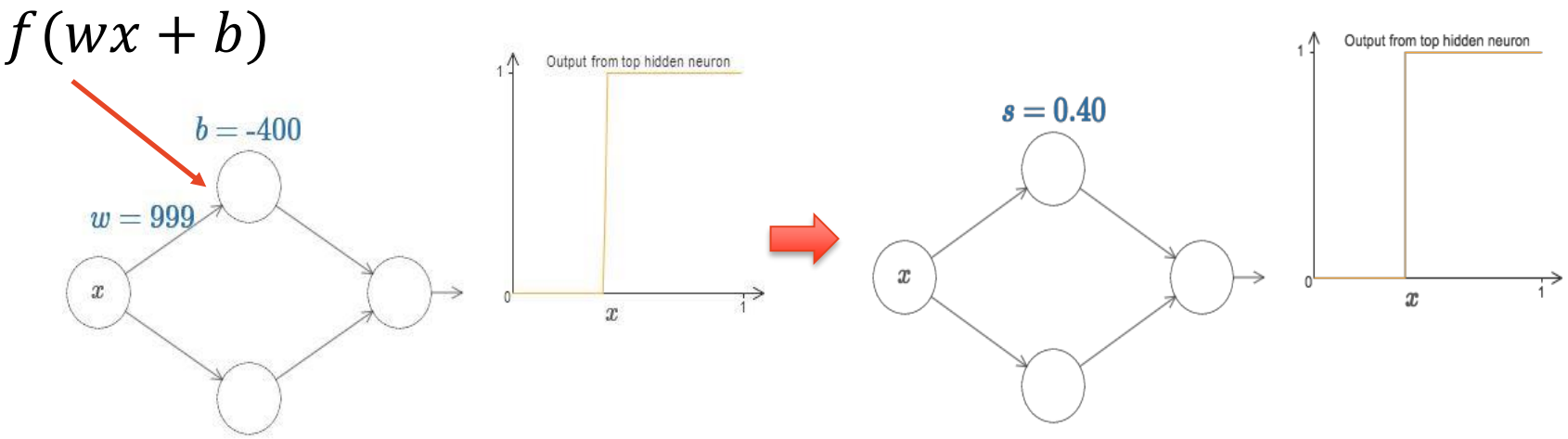


Image credit to: https://neuralnetworksanddeeplearning.com

- Increase the weight so that the output is a very good approximation of a step function.
- The step is at position $s = -b/w$.
- We simplify the problem by describing hidden neurons using just a single parameter s.

Expressive power of a three-layer neural network

- A visual explanation on one input and one output functions

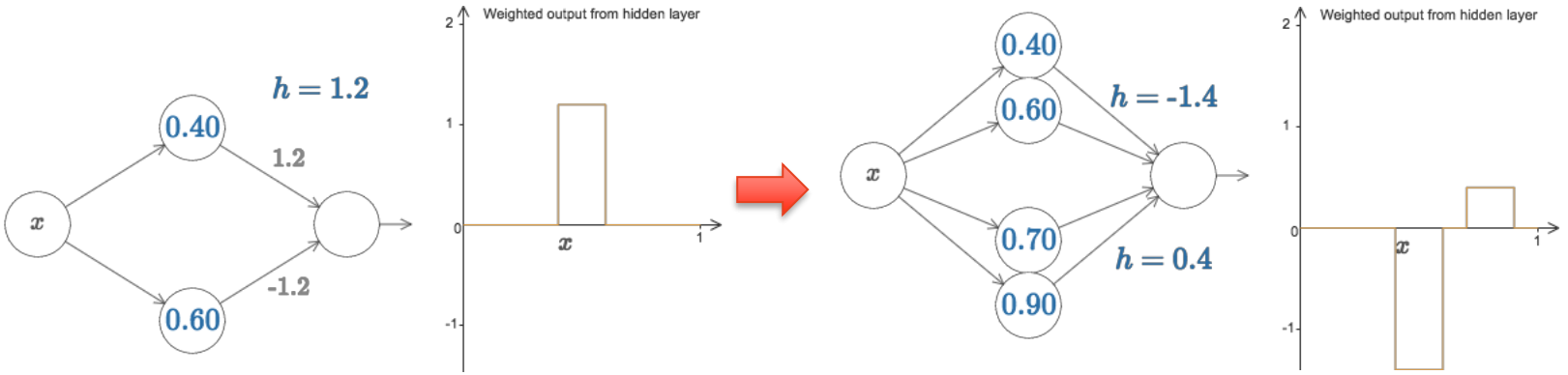


Image credit to: https://neuralnetworksanddeeplearning.com

- *Left*: Setting the weights of output to 1.2 and -1.2, we get a 'bump' function, which starts at 0.4, ends at 0.6 and has height 1.2.
- *Right*: By adding another pair of hidden neurons, we can get more bumps in the same network.

Expressive power of a three-layer neural network

- A visual explanation on one input and one output functions

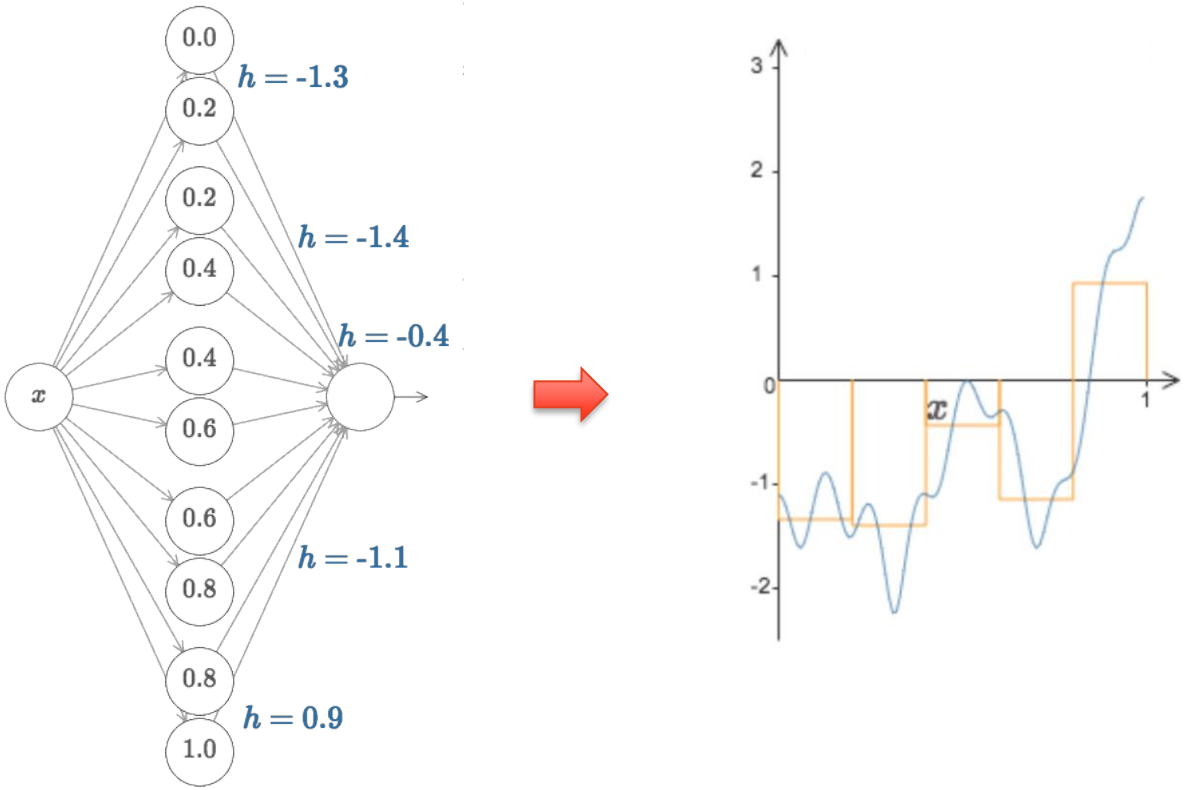
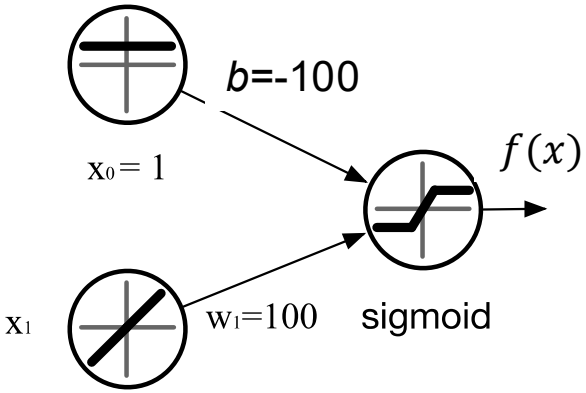


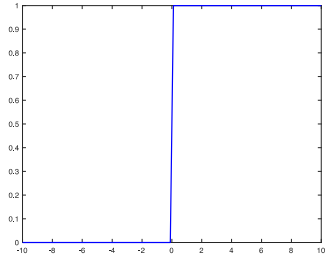
Image credit to: https://neuralnetworksanddeeplearning.com

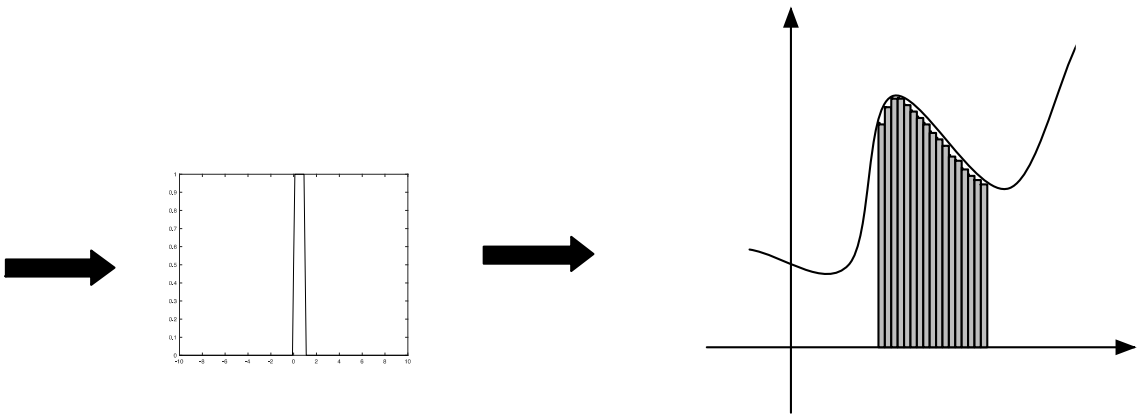
- By increasing the number of hidden neurons, we can make the approximation much better.

Expressive power of a three-layer neural network

– Overall procedure



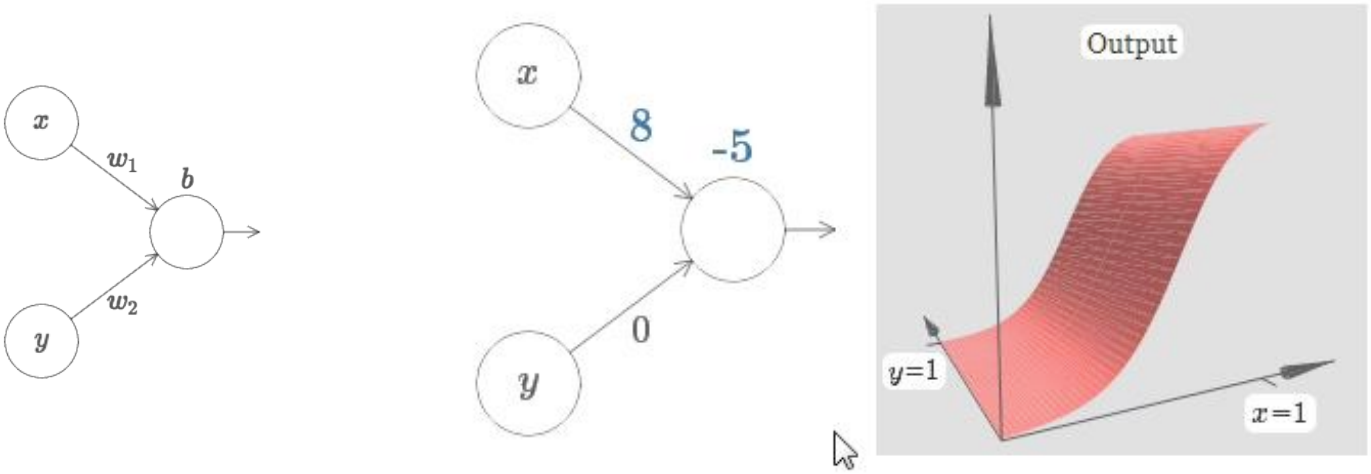




Expressive power of a three-layer neural network

- How about multiple input functions? The two input case:

  Fix $w_2 = 0$, change w_1 and b:

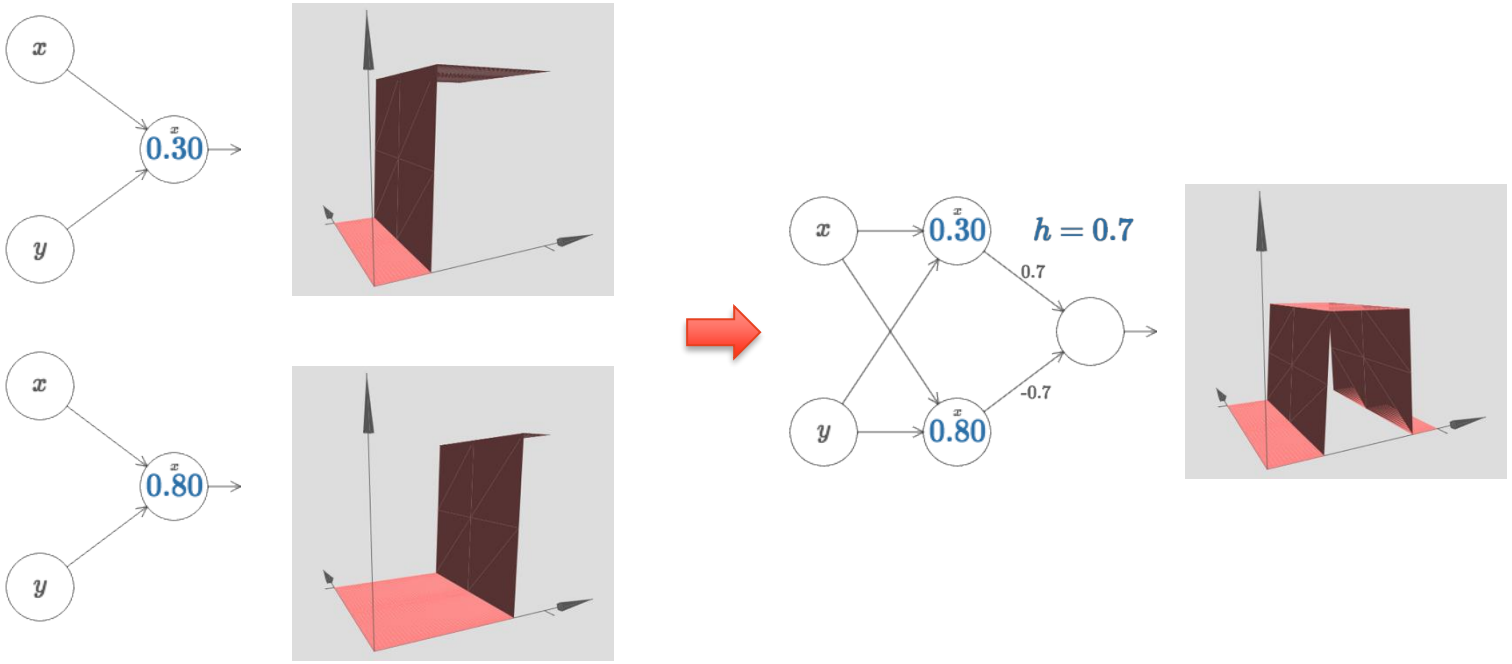


Video credit to: https://neuralnetworksanddeeplearning.com

  - Similar to one input case, the weight changes the shape and the bias changes the position.
  - The step point: $s_x = -b/w_1$

Expressive power of a three-layer neural network

– How about multiple input functions? The two input case:



Video credit to: https://neuralnetworksanddeeplearning.com

Expressive power of a three-layer neural network

– How about multiple input functions? The two input case:

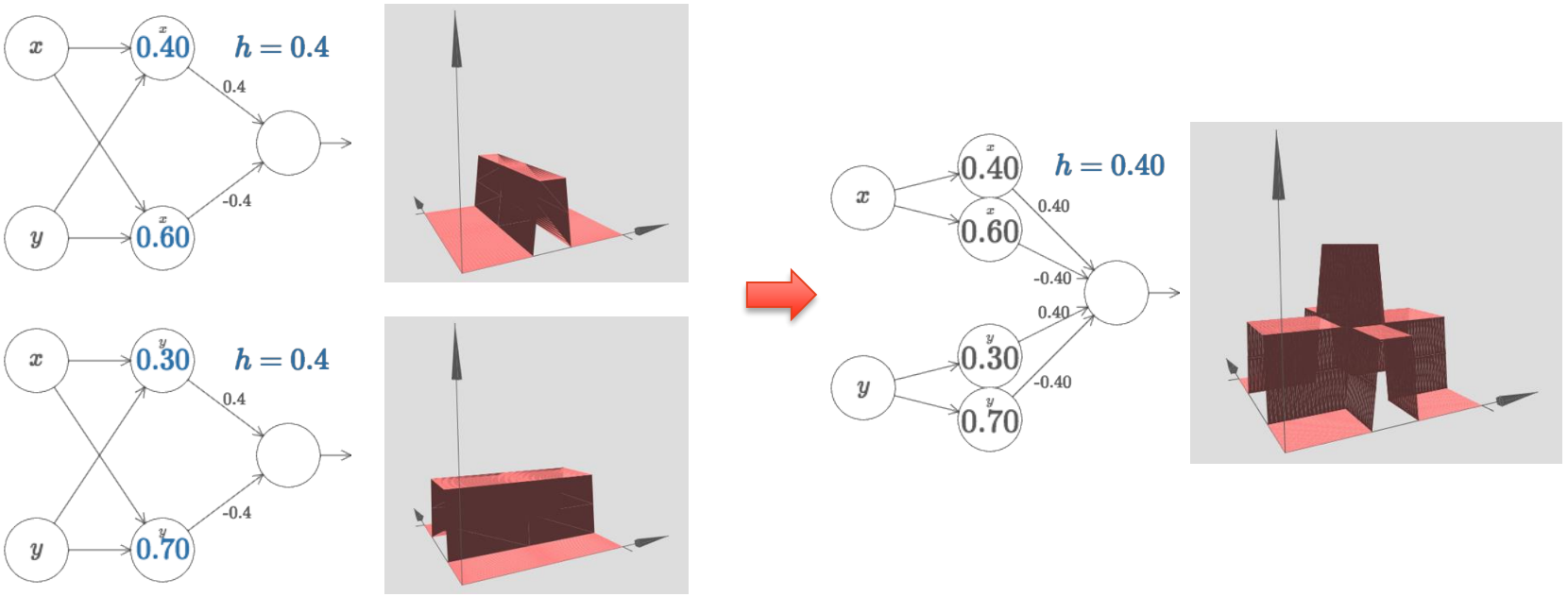


Image credit to: https://neuralnetworksanddeeplearning.com

Expressive power of a three-layer neural network

- How about multiple input functions? The two input case:

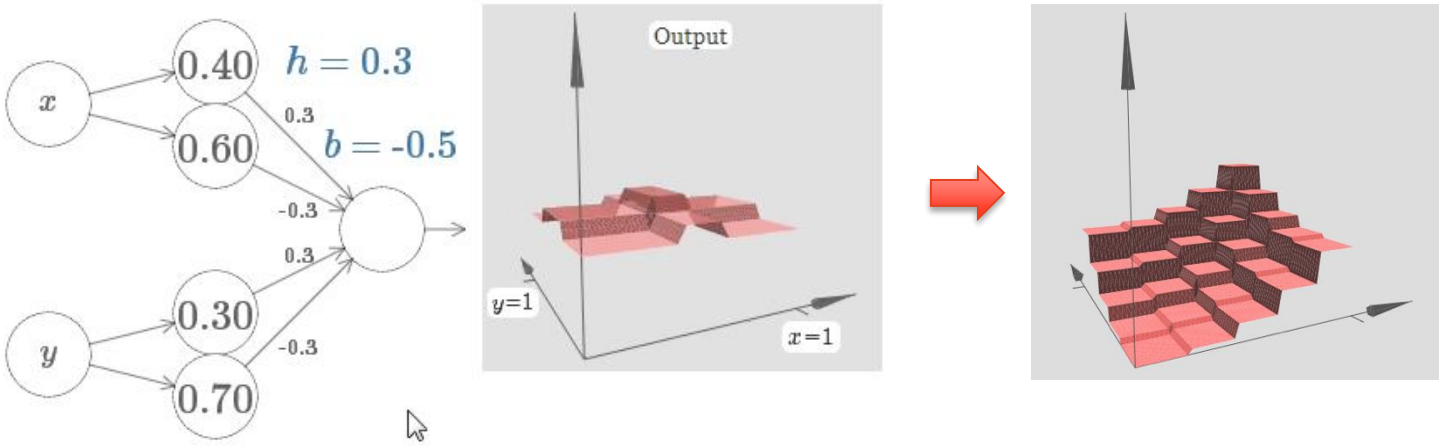


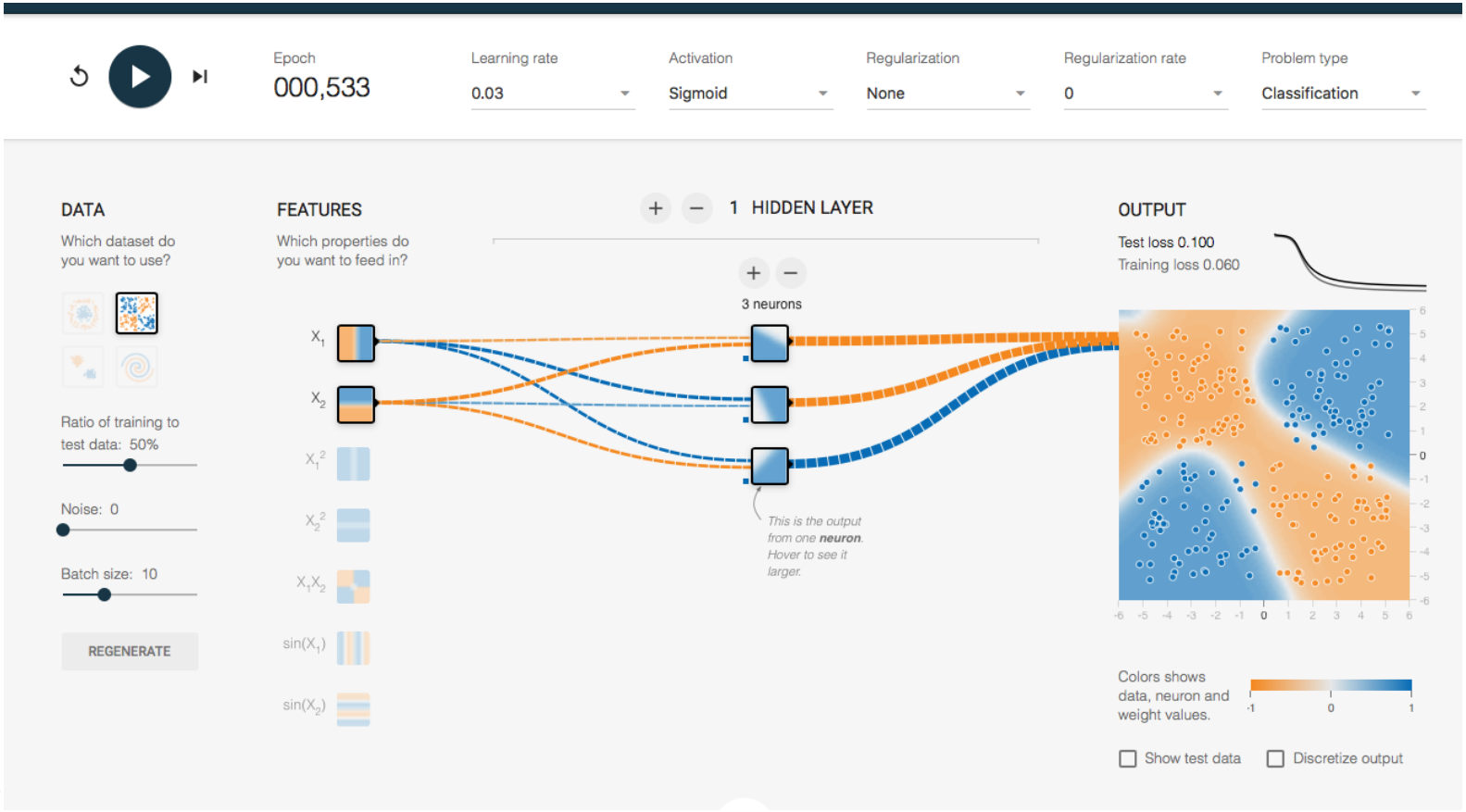
Image credit to: https://neuralnetworksanddeeplearning.com

- By adding bias to the output neuron and adjusting the value of h and b, we can construct a 'tower' function.
- The same idea can be used to compute as many towers as we like. We can also make them as thin as we like, and whatever height we like. As a result, we can ensure that the weighted output from the second hidden layer approximates any desired function of two variables.

A toy example

http://playground.tensorflow.org/

1. Linear case without an hidden layer
2. Nonlinear case with a hidden layer

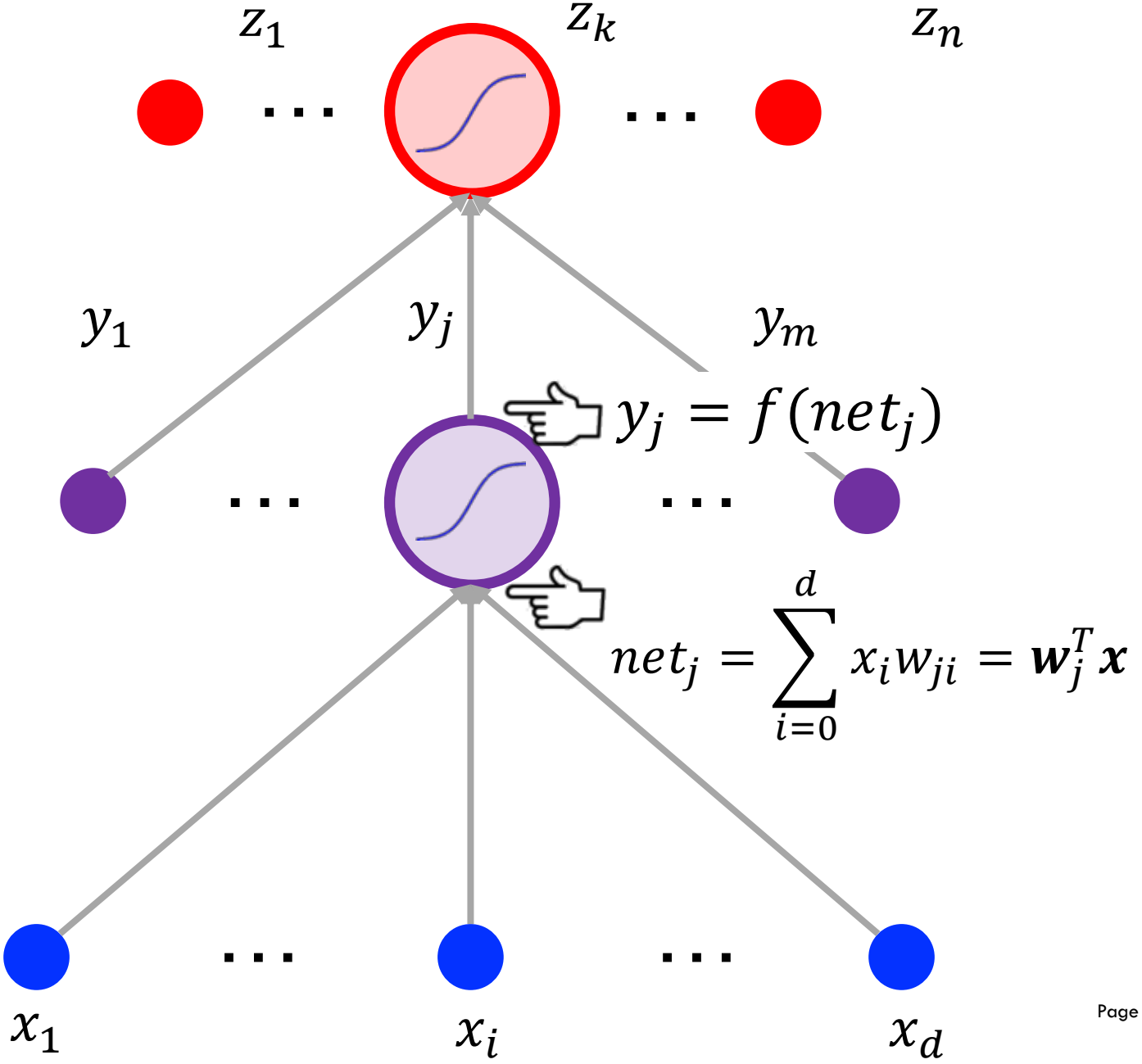


How does a neural network learn?

Neural Network Learn from Errors

1. The Neural Network give some outputs
2. Calculate the loss (error) by comparing neural network outputs with target outputs with Loss Functions
3. Backpropagate the error throughout the neural network to calculate gradient.
4. Update Neural Network parameters with Gradient Descent

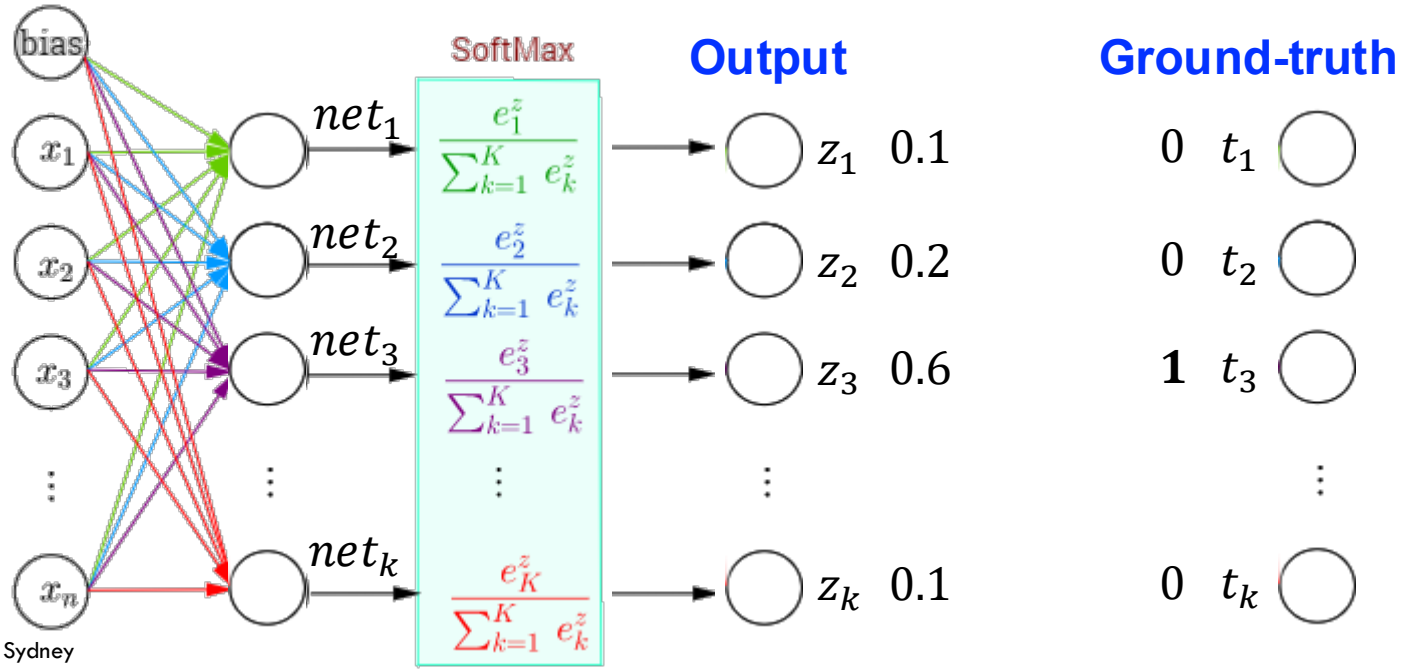
Recall the feedforward calculation



Softmax Function

- In multi-class classification, **softmax function** before the output layer is to assign conditional probabilities (given x) to each one of the K classes.

$$\hat{P}(class_k|x) = z_k = \frac{e^{net_k}}{\sum_{i=1}^{K} e^{net_i}}$$



Cross Entropy Loss

– Given two probability distributions $\boldsymbol{t}$ and $\boldsymbol{z}$,

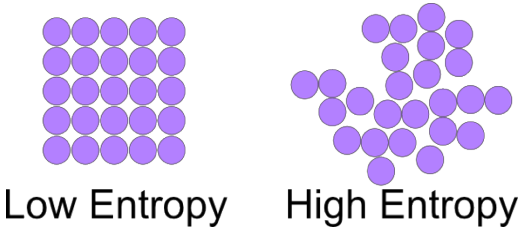
$$CrossEntropy(\boldsymbol{t}, \boldsymbol{z}) = -\sum_i t_i \log z_i$$

$$= -\sum_i t_i \log t_i + \sum_i t_i \log t_i - \sum_i t_i \log z_i$$

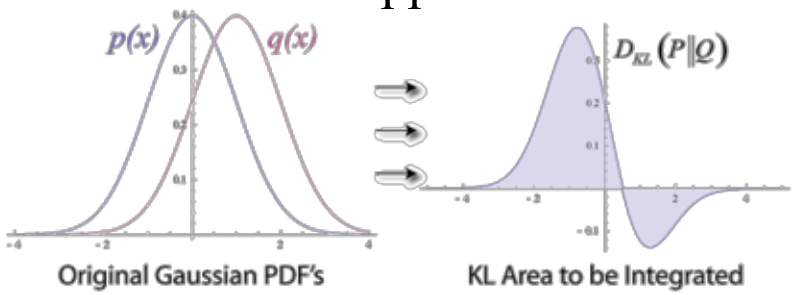
$$= -\sum_i t_i \log t_i + \sum_i t_i \log \frac{t_i}{z_i}$$

$$= Entropy(\boldsymbol{t}) + D_{KL}(\boldsymbol{t}|\boldsymbol{z})$$

Entropy is a measure of degree of disorder or randomness.



KL (Kullback-Leibler) divergence is a measure of the information lost when Z is used to approximate T



Training error

- Euclidean distance.

$$J(t,z) = \frac{1}{2}\sum_{k=1}^{C}(t_k - z_k)^2 = \frac{1}{2}\|\boldsymbol{t} - \boldsymbol{z}\|^2$$

- If both $\{t_k\}$ and $\{z_k\}$ are probability distributions, we can use cross entropy.

$$J(t,z) = -\sum_{k=1}^{C} t_k \log z_k$$

- Cross entropy is asymmetric. In some cases we may use this symmetric form.

$$J(t,z) = -\sum_{k=1}^{C}(t_k \log z_k + z_k \log t_k)$$

Backpropagation

Gradient descent

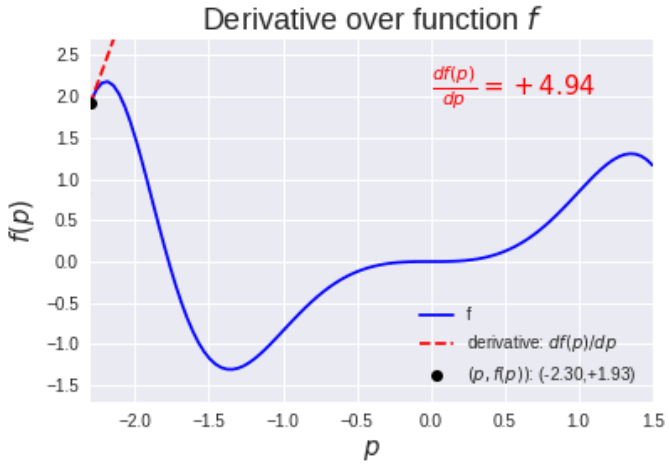
– Weights are initialized with random values, and then are updated in a direction reducing the error.

$$\Delta \boldsymbol{w} = -\eta \frac{\partial J}{\partial \boldsymbol{w}}$$

where η is the learning rate.

Iteratively update

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) + \Delta \boldsymbol{w}(t)$$

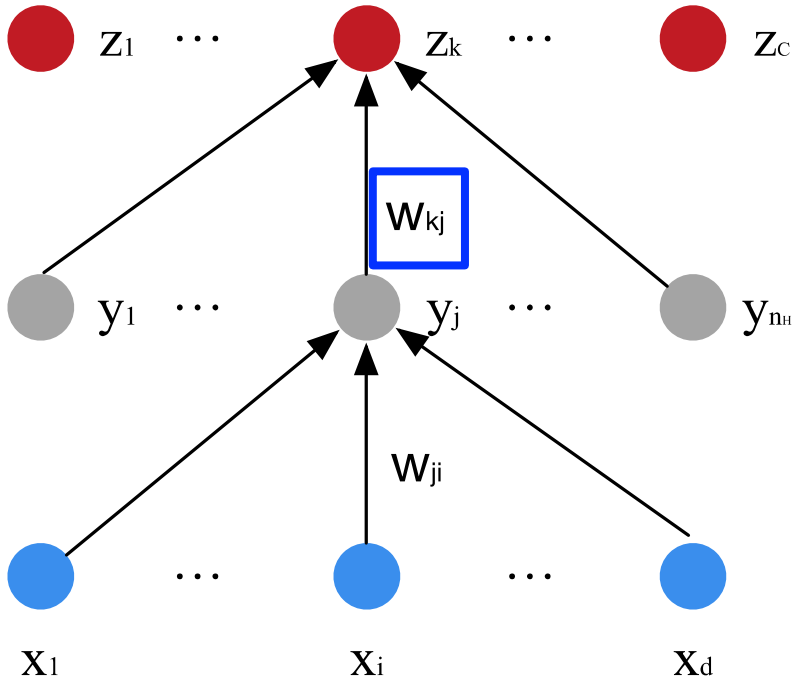


Hidden-to-output weight w_{kj}

– Chain rule

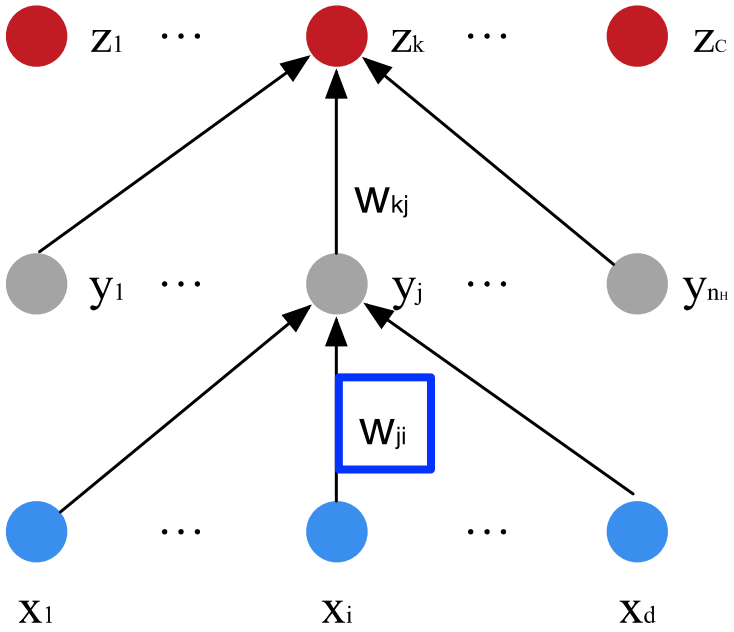
$$\frac{\partial J}{\partial w_{kj}} = \frac{\partial J}{\partial net_k}\frac{\partial net_k}{\partial w_{kj}} = \frac{\partial J}{\partial net_k} y_j$$

$$\begin{cases} J(\boldsymbol{w}) = \frac{1}{2}\|\boldsymbol{t} - \boldsymbol{z}\|^2 \\ z_k = f(net_k) \\ net_k = \sum_{j=1}^{n_H} y_j w_{kj} + w_{k0} \end{cases}$$



Input-to-hidden weight w_{ji}

$$\frac{\partial J}{\partial w_{ji}} = \frac{\partial J}{\partial y_j}\frac{\partial y_j}{\partial net_j}\frac{\partial net_j}{\partial w_{ji}}$$



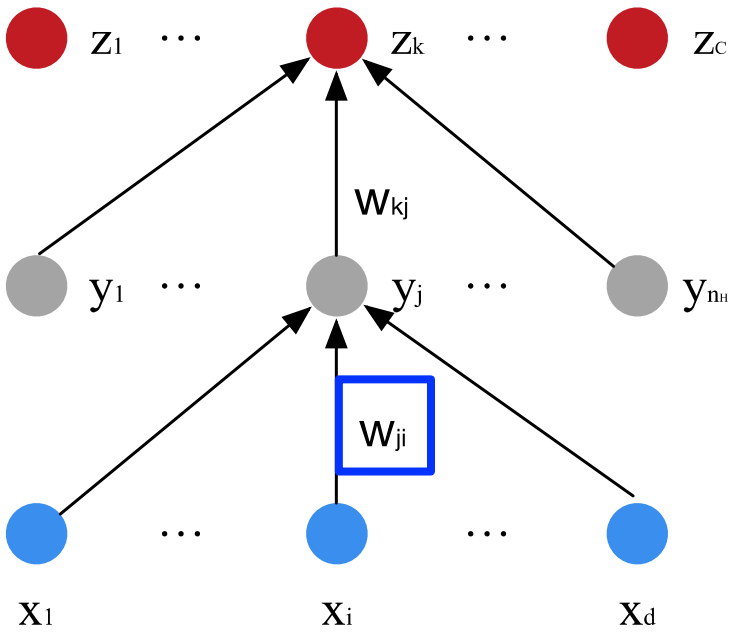
$$\frac{\partial J}{\partial y_j} = \sum_{k=1}^{c}\frac{\partial J}{\partial z_k}\frac{\partial z_k}{\partial y_j}$$

$$net_j = \sum_{i=1}^{d} x_j w_{ji} + w_{j0}$$

$$y_j = f(net_j)$$

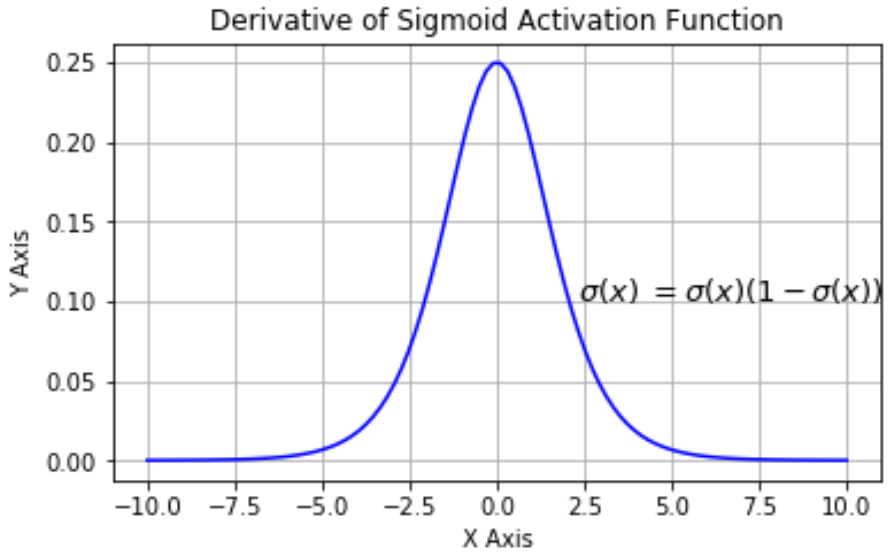
Vanishing gradients problem

$$\frac{\partial J}{\partial w_{ji}} = \frac{\partial J}{\partial y_j}\frac{\partial y_j}{\partial net_j}\frac{\partial net_j}{\partial w_{ji}}$$



$$\frac{\partial y_j}{\partial net_j} = \sigma'\left(\sum_{i=1}^{d} x_j w_{ji} + w_{j0}\right)$$

$$\sigma'(x) = \sigma(x)(1-\sigma(x)) = \frac{e^{-x}}{(e^{-x}+1)^2}$$

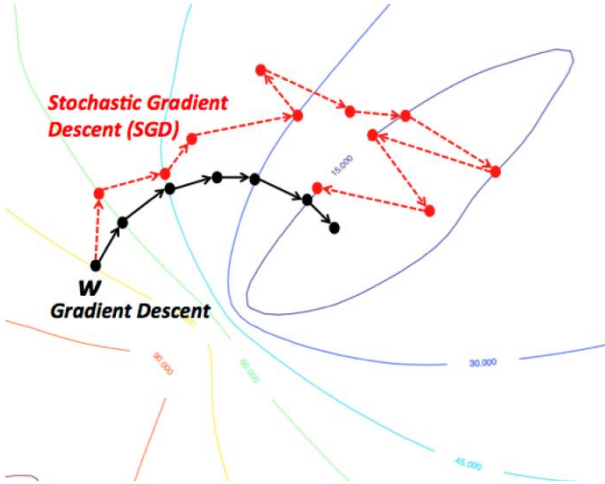


Stochastic Gradient Descent

Stochastic Gradient Descent (SGD)

- Given n training examples, the target function can be expressed as

$$J(\boldsymbol{w}) = \sum_{p=1}^{n} J_p(\boldsymbol{w})$$



- Batch gradient descent

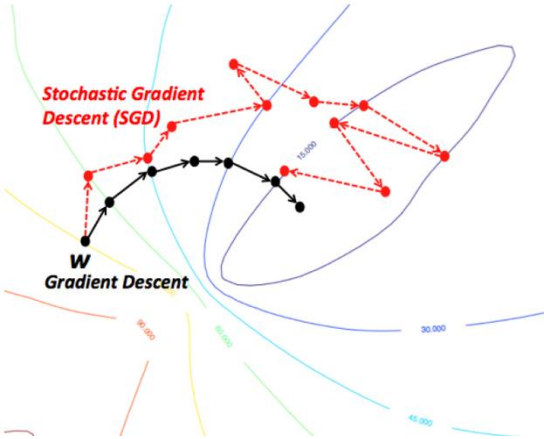
$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \frac{\eta}{n} \sum_{p=1}^{n} \nabla J_p(\boldsymbol{w})$$

- In SGD, the gradient of $J(\boldsymbol{w})$ is approximated on a single example or a mini-batch of examples

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla J_p(\boldsymbol{w})$$

One-example based Backpropagation

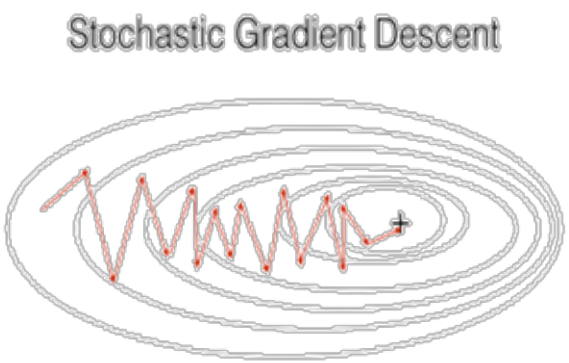
- Algorithm 1 (one-example based BP)
  - 1 **begin initialize** network topology (n_H), $\boldsymbol{w}$, criterion θ, η, $m \leftarrow 0$
  - 2 **do** $m \leftarrow m + 1$
  - 3 $x^m \leftarrow$ randomly chosen pattern
  - 4 $w_{ji} \leftarrow w_{ji} + \eta\delta_j x_i; \quad w_{kj} \leftarrow w_{kj} + \eta\delta_k y_j$
  - 5 **until** $\nabla J(\boldsymbol{w}) < \theta$
  - 6 **return** $\boldsymbol{w}$
  - 7 **end**

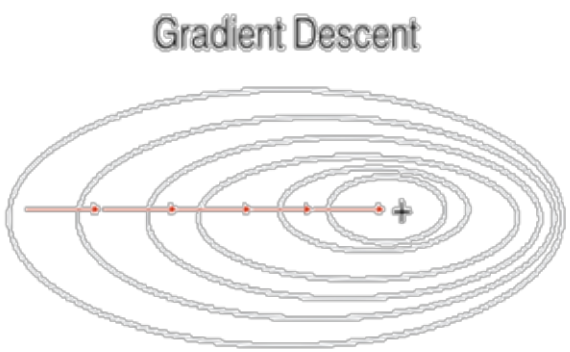


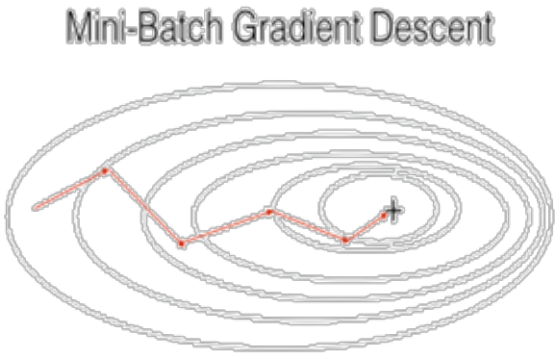
- In one-example based training, a weight update may reduce the error on the single pattern being presented, yet increase the error on the full training set.

Mini-batch based SGD

- Divide the training set into mini-batches.
- In each epoch, randomly permute mini-batches and take a mini-batch sequentially to approximate the gradient
  - One epoch is usually defined to be one complete run through all of the training data.
- The estimated gradient at each iteration is more reliable.







Mini-Batch Backpropagation

- Algorithm 2 (mini-batch based BP)
  - 1 **begin initialize** network topology (n_H), $\boldsymbol{w}$, criterion θ, η, r $\leftarrow 0$
  - 2 **do** $r \leftarrow r + 1$ (increment epoch)
  - 3 $m \leftarrow 0$; $\Delta w_{ji} \leftarrow 0$; $\Delta w_{kj} \leftarrow 0$;
  - 4 **do** $m \leftarrow m + 1$
  - 5 $x^m \leftarrow$ randomly chosen pattern
  - 6 $\Delta w_{ji} \leftarrow \Delta w_{ji} + \eta \delta_j x_i$; $\Delta w_{kj} \leftarrow \Delta w_{kj} + \eta \delta_k y_j$
  - 7 **until** $m = n$
  - 8 $w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$; $w_{kj} \leftarrow w_{kj} + \Delta w_{kj}$
  - 9 **until** $\nabla J(\boldsymbol{w}) < \theta$
  - 10 **return** $\boldsymbol{w}$
  - 11 **end**

SGD Analysis

- One-example based SGD
  - Estimation of the gradient is noisy, and the weights may not move precisely down the gradient at each iteration
  - Faster than batch learning, especially when training data has redundancy
  - Noise often results in better solutions
  - The weights fluctuate and it may not fully converge to a local minimum
- Min-batch based SGD
  - Conditions of convergence are well understood
  - Some acceleration techniques only operate in batch learning
  - Theoretical analysis of the weight dynamics and convergence rates are simpler

Summarization

- What is a neural network?

  A neural network is a model made of layers of simple neurons that map inputs to outputs.

- What does a neural network learn?

  A neural network learns a set of parameters (weights and biases) that define this mapping.

- How does a neural network learn?

  It learns by adjusting those parameters to reduce a loss using gradient-based optimization (backpropagation + gradient descent).

Optimization for Training Deep Models

THE UNIVERSITY OF SYDNEY

A common training process for neural networks

1. Initialize the parameters

2. Choose an optimization algorithm

3. Repeat these steps:
    1. Forward propagate an input
    2. Compute the cost function
    3. Compute the gradients of the cost with respect to parameters using backpropagation
    4. Update each parameter using the gradients, according to the optimization algorithm

Outline

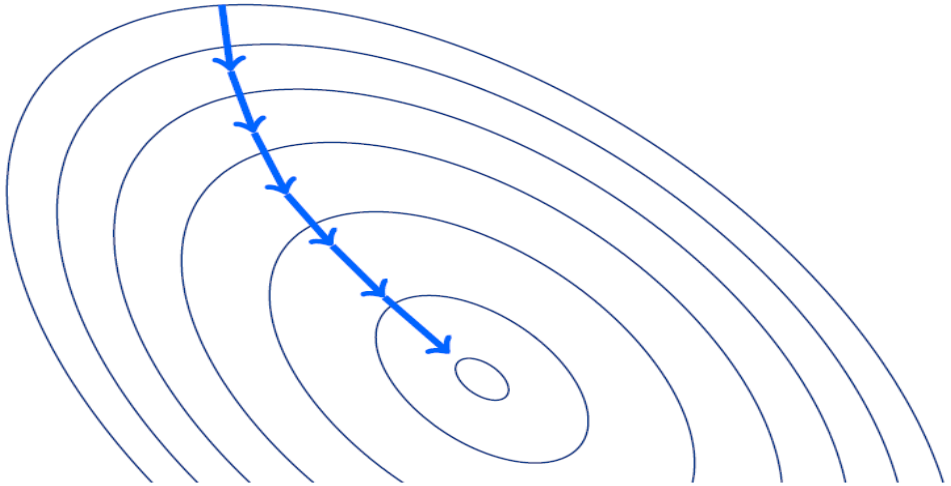
1. Gradient Descent Optimization
   - Gradient descent
   - Momentum
   - Adam
2. Initialization
   - Xavier
   - Kaiming

Gradient descent

- $J(\theta, \mathcal{X})$ is the **objective function**, θ are the parameters, $\mathcal{X}$ is the training dataset. We want to optimize $J(\theta, \mathcal{X})$ by:

$$\theta = \theta - \eta \cdot \nabla_\theta J(\theta, \mathcal{X})$$

where $\nabla_\theta J(\theta, \mathcal{X})$ is the gradient of objective function w.r.t. the parameters. η is the learning rate.



Gradient descent

- **Batch gradient descent:**

  Compute the gradient for the **entire** training dataset.

$$\theta = \theta - \eta \cdot \nabla_\theta J(\theta, \mathcal{X}^{(1:end)})$$

- **Stochastic gradient descent:**

  Compute the gradient for **each** training example.

$$\theta = \theta - \eta \cdot \nabla_\theta J(\theta, \mathcal{X}^{(i)})$$

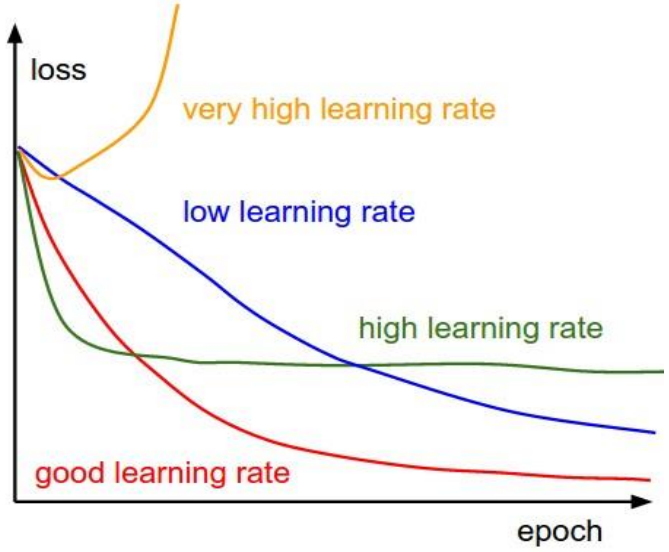
- **Mini-batch gradient descent:**

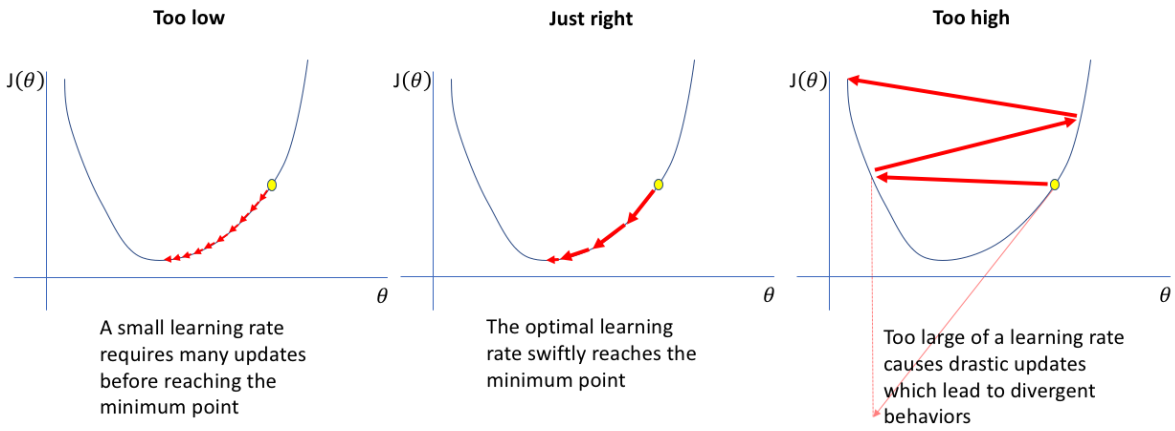
  Compute the gradient for every **mini-batch** training examples.

$$\theta = \theta - \eta \cdot \nabla_\theta J(\theta, \mathcal{X}^{(i:i+n)})$$

Some Challenges For Gradient descent

- **Choosing a proper learning rate can be difficult.**
  - A learning rate that is too small leads to painfully slow convergence.
  - A learning rate that is too large can hinder convergence and cause the loss function to fluctuate around the minimum or even to diverge.

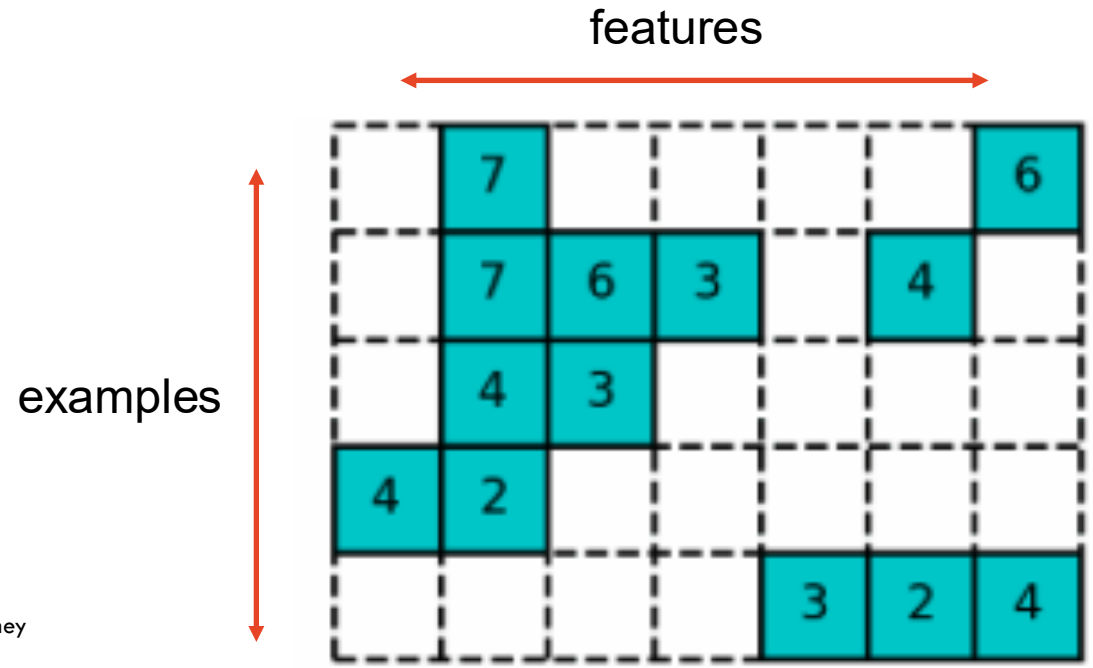




http://srdas.github.io/DLBook/GradientDescentTechniques.html

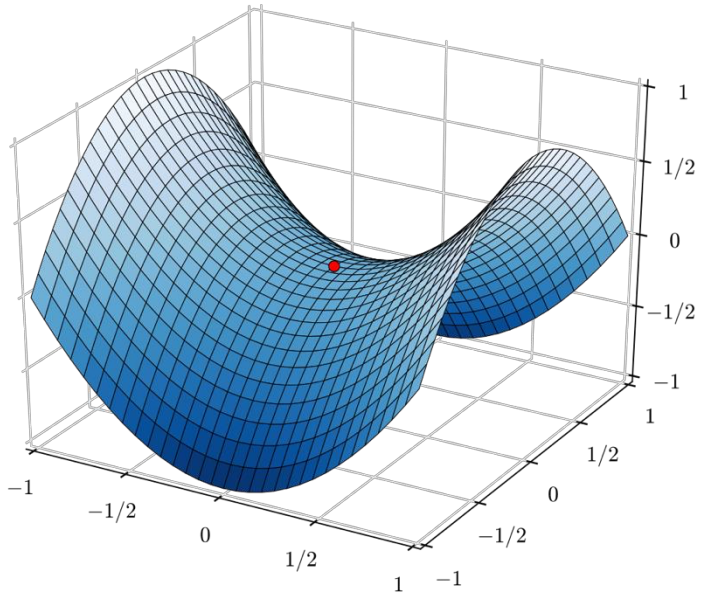
Some Challenges For Gradient descent

- **The same learning rate applies to all parameter updates.**
  - If our data is sparse and our features have very different frequencies, we might not want to update all of them to the same extent, but perform a larger update for rarely occurring features.



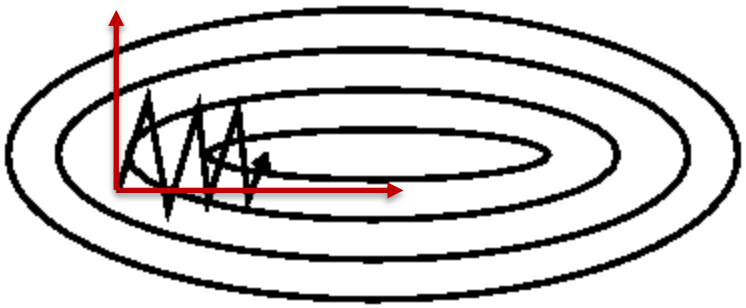
Some Challenges For Gradient descent

- **Easily get trapped in numerous saddle points.**
  - Saddle points are points where one dimension slopes up and another slopes down. These saddle points are usually surrounded by a plateau of the same error, which makes it notoriously hard for SGD to escape, as the gradient is close to zero in all dimensions.

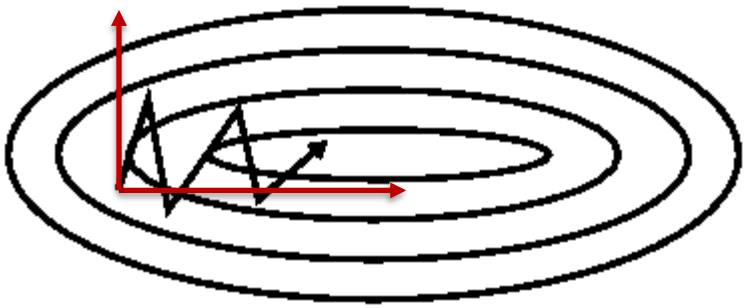


Momentum

SGD has trouble navigating ravines, i.e. areas where the surface curves much more steeply in one dimension than in another, which are common around local optima.



Without Momentum



With Momentum

https://www.willamette.edu/~gorr/classes/cs449/momrate.html

Momentum tries to accelerate SGD in the relevant direction and dampens oscillations.

Momentum

The momentum term increases for dimensions whose gradients point in the same directions and reduces updates for dimensions whose gradients change directions.

$$v_t = \gamma v_{t-1} + \eta \nabla_\theta J(\theta)$$
$$\theta_t = \theta_{t-1} - v_t$$

Momentum term γ is usually set to 0.9 or a similar value.

As a result, we gain faster convergence and reduced oscillation.

Adaptive Learning Rate Methods

Motivation

- Till now we assign the same learning rate to all features.
- If the feature varies in importance and frequency, why is this a good idea?
- It's probably not!

Adam

Store an exponentially decaying average of past squared gradients for adaptive learning rate

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Keep an exponentially decaying average of past gradients for momentum.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Adam - bias-correct

Since m_t and v_t are initialized as vectors of 0's, they are biased towards zero, especially during the initial time steps or when the decay rates are small (i.e. close to 1).

$$\begin{cases} v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\ m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \end{cases}$$

$$\begin{cases} \hat{m}_t = \dfrac{m_t}{1-\beta_1^t} \\ \hat{v}_t = \dfrac{v_t}{1-\beta_2^t} \end{cases} \longrightarrow \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

Parameter Initialization for Deep Models

THE UNIVERSITY OF SYDNEY

A common training process for neural networks

1. Initialize the parameters

2. Choose an optimization algorithm

3. Repeat these steps:
    1. Forward propagate an input
    2. Compute the cost function
    3. Compute the gradients of the cost with respect to parameters using backpropagation
    4. Update each parameter using the gradients, according to the optimization algorithm

Outline

1. Gradient Descent Optimization
   - Gradient descent
   - Momentum
   - Adam
2. **Initialization**

The initialization step can be critical to the model's ultimate performance, and it requires the right method.

Initialization

- All zero initialization
- Small random numbers
- Calibrating the variances

Initialization

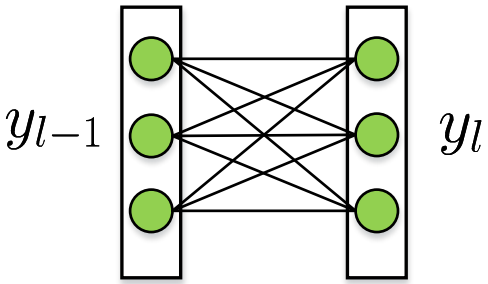
All zero initialization:

Every neuron computes the same output.

They will also compute the same gradients.

They will undergo the exact same parameter updates.

There will be no difference between all the neurons.



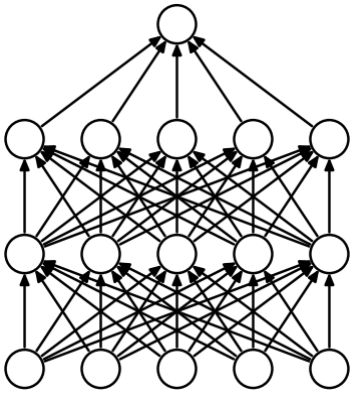
$$y_l = W y_{l-1} + b, \quad \frac{\partial \mathcal{L}}{\partial y_{l-1}} = W^\top \frac{\partial \mathcal{L}}{\partial y_l}$$
$$if \quad W = 0, \quad then \quad \frac{\partial \mathcal{L}}{\partial y_{l-1}} = 0$$
$$then, \quad \frac{\partial \mathcal{L}}{\partial y_m} = 0, \quad m = 0, ...l-1.$$

Initialize weights with values too small or too large

Despite breaking the symmetry, initializing the weights with values (i) too small or (ii) too large leads respectively to (i) slow learning or (ii) divergence.

Assuming all the activation functions are linear (identity function), we have

$$\hat{y} = W^{[L]}W^{[L-1]}W^{[L-2]} \ldots W^{[3]}W^{[2]}W^{[1]}x$$



Case 1: A too-large initialization leads to exploding gradients

Case 2: A too-small initialization leads to vanishing gradients

How to find appropriate initialization values

1. The *mean* of the activations should be zero.
2. The *variance* of the activations should stay the same across every layer.

Under these two assumptions, the backpropagated gradient signal should not be multiplied by values too small or too large in any layer. It should travel to the input layer without exploding or vanishing.

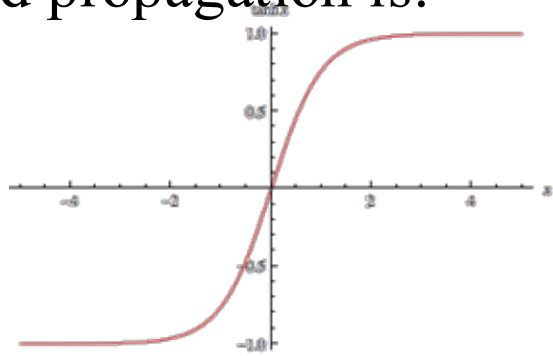
Initialization (Xavier) -- [Xavier Glorot, Yoshua.Bengio 2010]

How Xavier Initialization keeps the variance the same across every layer?

Assume the activation function is *tanh*. The forward propagation is:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = \tanh(z^{[l]})$$



How should we initialize our weights such that $Var\left(a^{[l-1]}\right) = Var(a^{[l]})$?

Early on in the training, we are in the ***linear regime* of *tanh***. Values are small enough and thus $\tanh\left(z^{[l]}\right) \approx z^{[l]}$, meaning that

$$Var\left(a^{[l]}\right) = Var(z^{[l]})$$

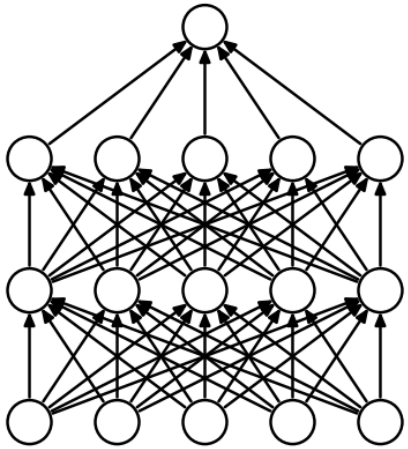
Initialization (Xavier) -- [Xavier Glorot, Yoshua.Bengio 2010]

How Xavier Initialization keeps the variance the same across every layer?

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = \tanh(z^{[l]})$$

$$Var\left(a^{[l]}\right) = Var\left(z^{[l]}\right)$$



$$z_k^{[l]} = \sum_{j=1}^{n^{[l-1]}} w_{kj}^{[l]} a_j^{[l-1]}$$

$$Var\left(a_k^{[l]}\right) = Var\left(z_k^{[l]}\right)$$

$$= Var\left(\sum_{j=1}^{n^{[l-1]}} w_{kj}^{[l]} a_j^{[l-1]}\right)$$

$$= \sum_{j=1}^{n^{[l-1]}} Var(w_{kj}^{[l]} a_j^{[l-1]})$$

Assumption

1. Weights are independent and identically distributed
2. Inputs are independent and identically distributed
3. Weights and inputs are mutually independent

Initialization (Xavier) -- [Xavier Glorot, Yoshua.Bengio 2010]

How Xavier Initialization keeps the variance the same across every layer?

$$z_k^{[l]} = \sum_{j=1}^{n^{[l-1]}} w_{kj}^{[l]} a_j^{[l-1]}$$

$$Var(XY) = E[X]^2 Var(Y) + Var(X)E[Y]^2 + Var(X)Var(Y)$$

$$Var\left(a_k^{[l]}\right) = Var\left(z_k^{[l]}\right)$$

$$= Var\left(\sum_{j=1}^{n^{[l-1]}} w_{kj}^{[l]} a_j^{[l-1]}\right)$$

$$= \sum_{j=1}^{n^{[l-1]}} Var(w_{kj}^{[l]} a_j^{[l-1]})$$

$$Var\left(w_{kj}^{[l]} a_j^{[l-1]}\right)$$
$$= E\left[w_{kj}^{[l]}\right]^2 Var\left(a_j^{[l-1]}\right) + Var\left(w_{kj}^{[l]}\right) E\left[a_j^{[l-1]}\right]^2$$
$$+ Var\left(w_{kj}^{[l]}\right) Var(a_j^{[l-1]})$$

Weights are initialized with zero mean, and inputs are normalized.

$$Var\left(a_k^{[l]}\right) = n^{[l-1]} Var\left(w_{kj}^{[l]}\right) Var(a_j^{[l-1]})$$

Initialization (Xavier) -- [Xavier Glorot, Yoshua.Bengio 2010]

How Xavier Initialization keeps the variance the same across every layer?

$$Var\left(a_k^{[l]}\right) = n^{[l-1]} Var\left(w_{kj}^{[l]}\right) Var(a_j^{[l-1]})$$

$$Var\left(W^{[l]}\right) = \frac{1}{n^{[l-1]}}$$

This expression holds for every layer of our network:

$$Var\left(a^{[L]}\right) = n^{[L-1]} Var\left(W^{[L]}\right) Var\left(a^{[L-1]}\right)$$
$$= n^{[L-1]} Var\left(W^{[L]}\right) n^{[L-2]} Var\left(W^{[L-1]}\right) Var\left(a^{[L-2]}\right)$$
$$= \cdots = \left[\prod_{l=1}^{L} n^{[l-1]} Var\left(W^{[l]}\right)\right] Var(x)$$

Initialization (Xavier) -- [Xavier Glorot, Yoshua.Bengio 2010]

$$Var\left(a^{[L]}\right) = \left[\prod_{l=1}^{L} n^{[l-1]} Var\left(W^{[l]}\right)\right] Var(x)$$

$$n^{[l-1]} Var\left(W^{[l]}\right) \begin{cases} < 1 & \rightarrow \text{ Vanishing Signal} \\ = 1 & \rightarrow \ Var\left(a^{[L]}\right) = Var(x) \\ > 1 & \rightarrow \text{ Exploding Signal} \end{cases}$$

$$Var\left(W^{[l]}\right) = \frac{1}{n^{[l-1]}}$$

https://pytorch.org/docs/stable/nn.init.html#torch.nn.init.xavier_normal_

Initialization (Xavier) -- [Xavier Glorot, Yoshua.Bengio 2010]

We worked on activations computed during the forward propagation, and get

$$Var\left(W^{[l]}\right) = \frac{1}{n^{[l-1]}}$$

The same result can be derived for the backpropagated gradients:

$$Var\left(W^{[l]}\right) = \frac{1}{n^{[l]}}$$

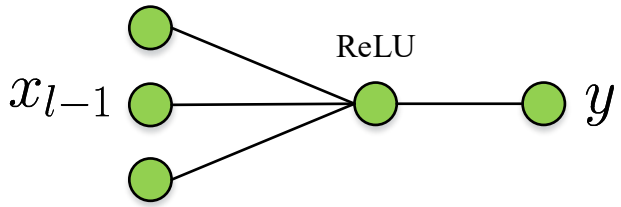
Xavier initialization would either initialize the weights as

xavier_normal $\quad \mathcal{N}(0, \frac{2}{n^{[l-1]} + n^{[l]}})$

xavier_uniform $\quad \mathcal{U}(-\frac{\sqrt{6}}{\sqrt{n^{[l-1]}+n^{[l]}}}, \frac{\sqrt{6}}{\sqrt{n^{[l-1]}+n^{[l]}}})$

Initialization (He) -- [Kaiming He, Xiangyu Zhang, et al. 2015]

If we use ReLU as the activation function, n is the number of inputs, then we have:



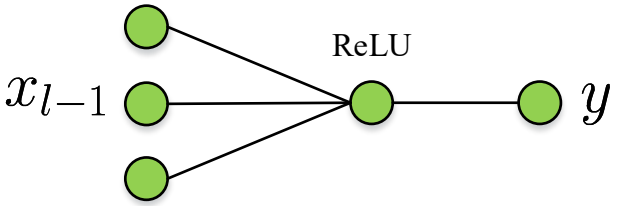
$$\begin{aligned}
\mathrm{Var}(y) &= \mathrm{Var}\left(\sum_i^n w_i x_i\right) \\
&= \sum_i^n \mathrm{Var}(w_i x_i) \\
&= \sum_i^n [E(w_i)]^2 \mathrm{Var}(x_i) + [E(x_i)]^2 \mathrm{Var}(w_i) + \mathrm{Var}(x_i)\mathrm{Var}(w_i) \\
&= \sum_i^n [E(x_i)]^2 \mathrm{Var}(w_i) + \mathrm{Var}(x_i)\mathrm{Var}(w_i) \\
&= \sum_i^n [E(x_i)]^2 \mathrm{Var}(w_i) + (E[x_i^2] - [E(x_i)]^2)\mathrm{Var}(w_i) \\
&= \sum_i^n E[x_i^2]\mathrm{Var}(w_i) = (n\mathrm{Var}(w))E[x_i^2]
\end{aligned}$$

$$Var(x) = E(x^2) - \big(E(x)\big)^2$$

Initialization (He) -- [Kaiming He, Xiangyu Zhang, et al. 2015]

If we use ReLU as the activation function, n is the number of inputs, then for layer l we have:

$$\mathrm{Var}(y_l) = (n\mathrm{Var}(w))\, E[x_{l-1}^2]$$
$$x_{l-1} = max(0, y_{l-1})$$



If y_{l-1} has zero mean, and has a symmetric distribution around zero:

$$E[x_{l-1}^2] = E[(max(0, y_{l-1}))^2] = \frac{1}{2}\mathrm{Var}[y_{l-1}]$$

So we have: $\mathrm{Var}(w) = \frac{2}{n}$

$$Var(x) = E(x^2) - \big(E(x)\big)^2$$

kaiming_normal $\mathcal{N}(0, \frac{2}{n})$

kaiming_uniform $\mathcal{U}(-\sqrt{\frac{6}{n}}, \sqrt{\frac{6}{n}})$

Thank you!